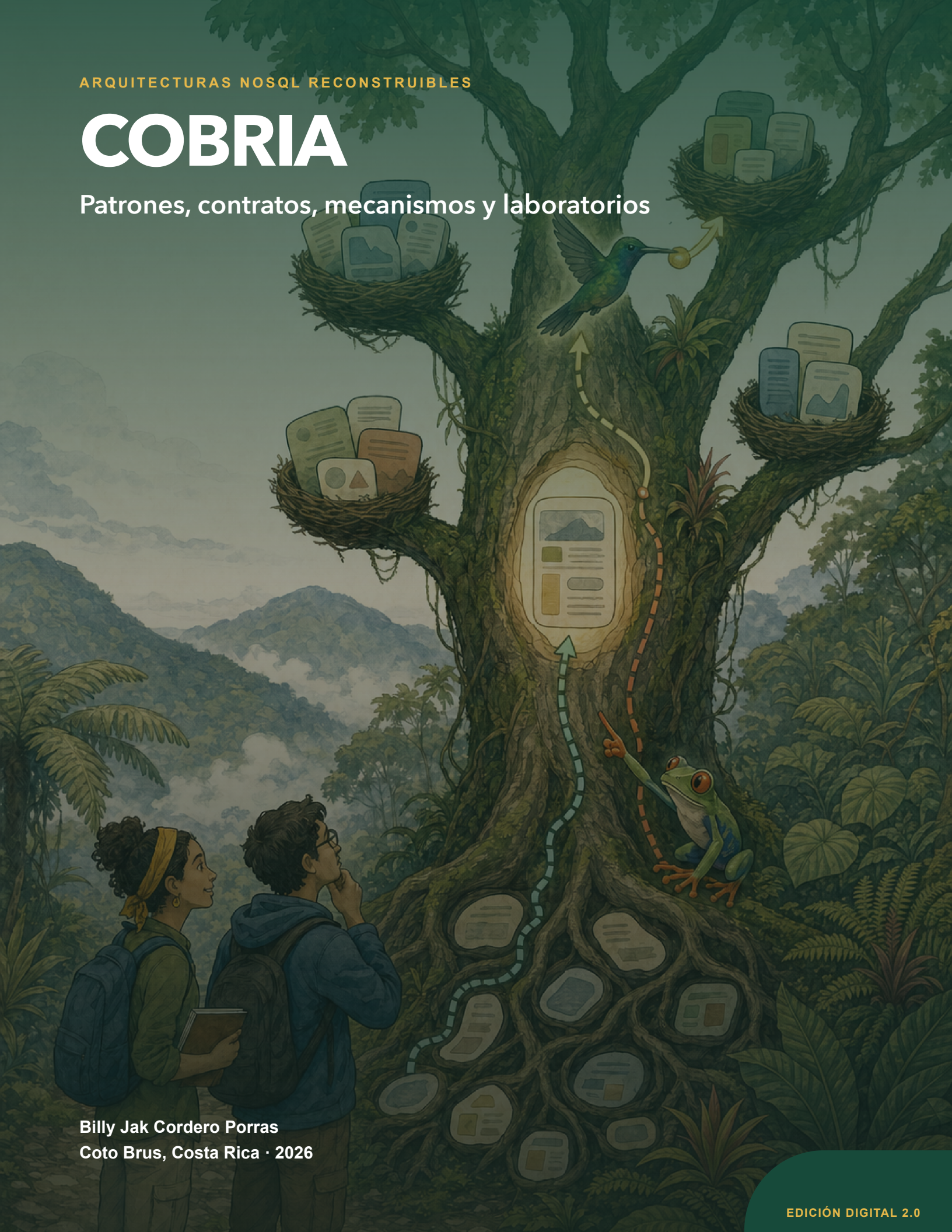
ARQUITECTURAS NOSQL RECONSTRUIBLES

COBRIA

Patrones, contratos, mecanismos y laboratorios



Billy Jak Cordero Porras
Coto Brus, Costa Rica · 2026

EDICIÓN DIGITAL 2.0

EDICIÓN COMPLETA 2.0

COBRIA

Arquitectura de software: catálogo pedagógico de patrones NoSQL reconstruibles

Billy Jak Cordero Porras
Coto Brus, Costa Rica · 2026

Libro, guía y laboratorio para aprender a diseñar datos documentales con autoridad, trazabilidad, derivados reconstruibles y ejemplos en TypeScript, Python, Java y Go.

INFORMACIÓN DE LA OBRA DIGITAL

Datos de esta edición

Título	COBRIA: Arquitectura de software
Subtítulo	Catálogo pedagógico de patrones NoSQL reconstruibles
Autor	Billy Jak Cordero Porras
Versión	Edición digital 2.0 revisada
Lugar y fecha	Coto Brus, Costa Rica, 2026
Idioma	Español
Formato	Documento pedagógico digital en PDF
Distribución	Publicación independiente y gratuita del autor

Uso y alcance. Material educativo de acceso libre para estudio, docencia, investigación y adaptación con atribución. No constituye una edición comercial ni un estándar certificado. COBRIA integra y formaliza principios existentes con contratos y mecanismos propios; no afirma que cada elemento sea una invención aislada. Los ejemplos requieren validación antes de emplearse en producción.

SINOPSIS

Una arquitectura que conserva el camino de regreso

COBRIA es una propuesta pedagógica para diseñar sistemas NoSQL documentales cuya información pueda explicarse, comprobarse y reconstruirse. Organiza treinta patrones alrededor de fuentes canónicas, límites de acceso, catálogos versionados, proyecciones de lectura, eventos idempotentes, analítica reproducible e inteligencia artificial con procedencia.

El libro combina analogías inspiradas en la naturaleza de Costa Rica, contratos verificables, decisiones arquitectónicas, laboratorios y ejemplos en TypeScript, Python, Java y Go. Está dirigido a estudiantes, docentes, personas desarrolladoras, arquitectos de software y equipos que necesitan optimizar lecturas sin perder autoridad, trazabilidad ni capacidad de auditoría.

Palabras clave

Arquitectura de software; NoSQL; bases de datos documentales; patrones de diseño; trazabilidad; reconstrucción; minería de datos; inteligencia artificial; gobernanza de datos.

Clasificación temática sugerida

Computación y tecnología · Ingeniería de software · Bases de datos · Arquitectura de sistemas.

SOBRE EL AUTOR

Billy Jak Cordero Porras

BJ

Billy Jak Cordero Porras es un joven cotobruseño aficionado a la informática y al desarrollo de software. Desde temprana edad ha construido sistemas para aprender y atender problemas cotidianos. Su curiosidad lo llevó a estudiar de forma autodidacta arquitectura de software, bases de datos, gestión de información e inteligencia artificial.

El trabajo con proyectos diversos lo enfrentó repetidamente al problema de conservar información comprensible, segura, trazable y útil. A partir de experimentación, comparación de patrones y revisión de errores formalizó COBRIA como investigación, artículo, libro y recurso pedagógico interactivo de acceso abierto.

Principio del autor. La investigación universitaria y la creación de soluciones deben producir conocimiento comprensible, verificable y de interés público.

NOTA DEL AUTOR

Un lenguaje para conversar sobre datos

COBRIA nació de observar una estructura repetida en sistemas reales: objetos que conservan reglas, repositorios que delimitan acceso, servicios que coordinan decisiones, catálogos que estabilizan significado y derivados que aceleran consultas o alimentan analítica.

Este libro no presenta cada mecanismo como una invención aislada. Reúne ideas conocidas, adaptaciones y contratos compuestos dentro de una propuesta evaluable. El lector puede adoptar una parte, medirla y rechazarla si no produce beneficio.

Antecedentes conceptuales

La propuesta dialoga con la tradición de patrones de diseño (Gamma et al., 1994), patrones de arquitectura empresarial (Fowler, 2002), diseño dirigido por el dominio (Evans, 2003), persistencia políglota NoSQL (Sadalage & Fowler, 2012) y sistemas intensivos en datos (Kleppmann, 2017). La organización de citas y referencias sigue los criterios generales de APA 7 (American Psychological Association, 2020).

Promesa pedagógica

Cada capítulo responde cinco preguntas: qué tensión existe, cómo se reconoce, qué responsabilidades introduce la solución, qué puede salir mal y qué evidencia demuestra su funcionamiento.

> **Alcance.** Todo el contenido se formula para almacenes NoSQL documentales, sin depender de un producto particular.

FUNDAMENTOS

Objeto, dato, servicio y catálogo

Objeto

Representa identidad y reglas que deben conservarse juntas.

Dato fechado

Registra valor, tiempo válido, tiempo conocido y procedencia.

Repositorio

Expresa el límite de acceso y evita consultas sin ámbito.

Servicio

Coordina una decisión que atraviesa varios objetos o mecanismos.

Catálogo

Estabiliza el significado de códigos, unidades y versiones.

Proyección

Optimiza una lectura y puede reconstruirse desde la autoridad.

FUNDAMENTOS

Patrón, contrato, mecanismo y decisión

Concepto	Pregunta	Evidencia
Patrón	¿Qué tensión recurrente resuelve?	Contexto, solución y consecuencias
Contrato	¿Qué debe cumplirse siempre?	Precondición, resultado y fallo
Mecanismo	¿Cómo se ejecuta?	Participantes y secuencia
Decisión	¿Por qué conviene aquí?	Alternativas, costo y métrica

COBRIA exige las cuatro capas. Un nombre sin contrato inspira; no gobierna. Un mecanismo sin decisión funciona; quizá no conviene.

ESPECIFICACIÓN CONGELADA

COBRIA Core 1.0

El núcleo normativo convierte el catálogo en una arquitectura comprobable.

Nivel	Obligaciones
C1 · Fundamental	Autoridad, ámbito, identidad, revisión y frontera de escritura.
C2 · Reconstruible	C1 más transformación versionada, equivalencia, idempotencia, reconstrucción y publicación segura.
C3 · Gobernado	C2 más procedencia, frescura, observabilidad, responsable y retiro.

Regla de conformidad. Una implementación solo declara un nivel cuando supera todas sus pruebas obligatorias. SQLite y PostgreSQL quedan fuera del alcance confirmatorio de Core 1.0.

PRECISIÓN CONCEPTUAL · CORE 1.1

Qué es COBRIA y qué no es

Una propuesta arquitectónica verificable, no una estructura de datos ni un producto de almacenamiento.

Núcleo

Invariantes y requisitos obligatorios.

Patrones

Soluciones recurrentes que ayudan a cumplirlos.

Mecanismos

Índices, proyecciones, manifiestos y revisiones.

Método

Inventariar, decidir, verificar, observar y retirar.

Adaptadores

Implementaciones intercambiables para motores NoSQL.

Evidencia

Pruebas, métricas, límites y resultados reproducibles.

Afirmación responsable. La contribución es la composición formalizada y comprobable de mecanismos conocidos; no la invención aislada de cada pieza.

CASO COMPLETO · CORE 1.2

Cuando desaparece el índice del bosque

Un solo recorrido conecta documento, contrato, fallo, reconstrucción, analítica e IA.

1. Una observación canónica declara identidad, ámbito, revisión y versión.
2. Un contrato produce una proyección de búsqueda sin autoridad propia.
3. La proyección activa se elimina de forma controlada.
4. Una candidata se reconstruye desde documentos y catálogo congelados.
5. La equivalencia se comprueba antes de cambiar el alias.
6. El corte analítico conserva partición temporal y manifiesto.
7. La recuperación para IA filtra ámbito antes de similitud y se abstiene sin evidencia.

Resultado. La búsqueda vuelve sin inventar datos; una revisión antigua no reemplaza la nueva y otro ámbito obtiene cero documentos.

APRENDER POR CONTRASTE

Ocho diseños que parecen funcionar

Proyección editable

Crea dos autoridades.

Ámbito al final

Filtra después de revelar candidatos.

Revisión obsoleta

Hace retroceder el estado visible.

Vector sin manifiesto

Oculta corpus, modelo y corte.

Fuga temporal

El futuro contamina la evaluación.

Reparar desde lo dañado

Propaga la corrupción.

Publicación destructiva

Deja lectores sin versión.

IA que siempre responde

Inventa cuando falta evidencia.

BANCO DE FALLOS · CORE 1.2

La arquitectura se aprende cuando algo se rompe

Se ejecutaron 300 pruebas locales: diez fallos, treinta repeticiones por escenario.

Familia	Escenarios	Resultado final
Pérdida	Eliminación y corrupción parcial	30/30 cada uno
Orden	Duplicado y revisión obsoleta	30/30 cada uno
Interrupción	25 %, 50 % y 75 %	30/30 cada uno
Evolución	Esquema y catálogo	30/30 tras corregir migración
Seguridad	Fuga de ámbito	Cero cruces

Hallazgo. La primera corrida descubrió que el cambio de esquema no estaba modelado. Corregirlo antes de publicar fue parte del resultado, no un dato que ocultar.

RUTAS DE LECTURA

Seis maneras de comenzar

Principiante

Caso → Canónico → Ámbito → Proyección → Reconstrucción.

Arquitectura

Modelo → Core → Fronteras → Presupuesto → Retiro.

NoSQL

Identidad → Revisión → Versiones → Publicación.

Minería

Dato fechado → Conjunto → Partición → Manifiesto.

IA

Recuperación → Evidencia → Abstención → Auditoría.

Conformidad

Requisitos → Amenazas → P1–A1 → D1–D8.

EVIDENCIA REPRODUCIBLE

Primera réplica NoSQL de Core 1.0

Se ejecutaron 180 corridas sobre PouchDB y LokiJS: dos motores documentales, tres escalas y treinta repeticiones por celda.

P1

Lecturas equivalentes; la proyección redujo la mediana en las seis celdas.

P2

Amplificación observada: dos escrituras por cambio proyectado.

R1

Reconstrucción total equivalente e idempotente.

S1

Cero documentos cruzados y rechazo sin ámbito.

A1

Manifiesto, procedencia y partición temporal verificables.

Límite

Evidencia embebida; no representa redes, regiones ni facturación administrada.

ESTADÍSTICA AMPLIADA

Leer más allá de la mediana

Core 1.1 informa p50, p95, p99, desviación absoluta mediana e intervalos bootstrap del 95 %.

Motor · N	R1 p50	R1 p95	IC95 de la mediana
LokiJS · 5 000	45,932 ms	50,859 ms	45,260–46,864 ms
PouchDB · 5 000	499,896 ms	521,530 ms	495,701–502,708 ms

No se eliminaron valores atípicos. Se utilizaron 4 000 remuestras y semilla registrada. La réplica distribuida posterior conserva por separado sus versiones, parámetros y resultados, de modo que las cifras embebidas no se confundan con las obtenidas mediante servicios.

> **Regla editorial.** Código disponible no significa evidencia ejecutada; cada afirmación debe señalar motor, versión, protocolo y fecha.

RÉPLICA DISTRIBUIDA · CORE 1.2

La misma prueba en tres servicios NoSQL

Se ejecutaron 270 corridas confirmatorias: MongoDB 8.0, CouchDB 3.4 y OpenSearch 2.19.3; tres escalas y treinta repeticiones por motor y escala.

Motor · N	Canónico p50	Proyección p50	R1 p50
MongoDB 8.0 · 5 000	157,856 ms	129,272 ms	3 179,603 ms
CouchDB 3.4 · 5 000	4 298,485 ms	1 477,093 ms	12 844,488 ms
OpenSearch 2.19.3 · 5 000	450,837 ms	250,173 ms	2 998,821 ms

Conformidad

P1, P2, R1, S1 y A1 aprobaron en las 270 corridas.

Aislamiento

No se observaron cruces de ámbito.

Comparación

Las latencias describen este entorno; no son una clasificación universal de motores.

Total confirmatorio

450 corridas al sumar las 180 embebidas y las 270 distribuidas.

TRANSPARENCIA EXPERIMENTAL

Un fallo de plataforma también es evidencia

OpenSearch 3.2.0 con Java 24 terminó por un fallo nativo al ejecutarse bajo emulación x86_64. La réplica confirmatoria se realizó con OpenSearch 2.19.3 y Java 21.

Lo que sí puede afirmarse

- el adaptador superó las pruebas con OpenSearch 2.19.3;
- el protocolo, los tamaños y las repeticiones quedaron registrados;
- la incidencia y la sustitución de versión permanecen visibles.

Lo que no puede afirmarse

- que OpenSearch 3.2.0 fue validado;
- que una sola máquina representa producción;
- que la comparación predice costos de nube o red.

Lección. La reproducibilidad no consiste en ocultar obstáculos, sino en separar con precisión lo ejecutado, lo fallido y lo pendiente.

SEGURIDAD POR DISEÑO

Doce amenazas que una proyección debe enfrentar

Datos y ámbitos

- confusión entre organizaciones;
- edición directa del derivado;
- repetición y revisión obsoleta;
- manipulación de proyecciones;
- eliminación incompleta;
- dependencia del proveedor.

Publicación, analítica e IA

- fallo durante publicación;
- envenenamiento analítico;
- recuperación no autorizada;
- evidencia insuficiente;
- herramientas excesivas;
- pérdida de procedencia.

Límite. COBRIA complementa, pero no reemplaza, autenticación, cifrado, respaldo o consenso del motor.

MAPA GENERAL

Nueve familias, una arquitectura

01
Modelado documental
6 patrones

02
Seguridad por ámbito
1 patrones

03
Catálogos y vocabularios
1 patrones

04
Proyecciones de lectura
2 patrones

05
Reconstrucción
4 patrones

06
Eventos y consistencia
6 patrones

07
Analítica reproducible
3 patrones

08
IA con procedencia
3 patrones

09
Gobernanza del ciclo de vida
4 patrones

RUTA DE APRENDIZAJE

Tres maneras de recorrer el atlas

Ruta esencial

Objeto canónico → Repositorio con ámbito → Catálogo versionado → Proyección de lectura → Reconstrucción total.

Ruta analítica

Dato fechado → Conjunto reproducible → Característica versionada → Partición sin fuga → Manifiesto.

Ruta de IA

Recuperación limitada → Abstención con evidencia → Herramienta gobernada → Auditoría narrativa.

REGLA DE LECTURA

De la analogía a la prueba

1. **Observar.** Reconozca el problema en una consulta, incidente o decisión.
2. **Delimitar.** Identifique autoridad, ámbito, revisión y consumidor.
3. **Contratar.** Escriba qué debe ocurrir y cómo falla.
4. **Instrumentar.** Conserve métricas y procedencia.
5. **Desafiar.** Pruebe repetición, atraso, interrupción y acceso prohibido.
6. **Retirar.** Simplifique cuando el beneficio desaparezca.

VOCABULARIO VISUAL

La metáfora del bosque documental

Las raíces representan documentos canónicos; el tronco, contratos; las ramas, servicios; los nidos, proyecciones; los senderos, ámbitos; y los colibríes, eventos que transportan cambios.

> La metáfora ayuda a recordar, pero la evidencia decide. Ninguna ilustración sustituye el contrato, la métrica o la prueba.

Fauna y flora

Las escenas emplean colibríes, ranas, tucanes, mariposas, perezosos y dantas junto con paisajes de Coto Brus.

01 Objeto canónico

Contrato compuesto | Evidencia: Adaptación COBRIA

La identidad, la validación y las versiones convierten entradas dispersas en documentos interpretables.

Propósito

Definir una representación autoritativa mínima y estable para la identidad y las reglas de una entidad.

Problema

Cuando varias pantallas, índices o integraciones escriben su propia versión de un hecho, ninguna copia puede explicar cuál manda. Una corrección se convierte en una reconciliación manual.

Analogía

Es el original firmado de un documento. Pueden existir fotocopias anotadas y resúmenes, pero una disputa vuelve al original.

Solución y contrato

El objeto canónico declara identidad, ámbito, revisión, versión de esquema y ciclo de vida. Las representaciones derivadas no se modifican como autoridad. Toda proyección apunta a su origen y puede comprobarse contra él.

Aplicación responsable

Úselo cuando hay reglas, múltiples consumidores o copias optimizadas. Evítelo como capa ceremonial en un CRUD pequeño. Empiece inventariando qué escritura posee autoridad y elimine rutas paralelas.

Métricas, costos y relaciones

Mida escrituras directas fuera de la frontera, entidades sin revisión y derivados sin origen. Se relaciona con Repositorio con ámbito, Proyección de lectura y Publicación por versión. El costo principal es diseñar migraciones y reglas de autoridad.

MECÁNICA · MODELADO DOCUMENTAL

Cómo funciona Objeto canónico

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
guardar(cambio) → validar → escribirCanónico → emitirRevisión
```

> **Invariante principal.** Definir una representación autoritativa mínima y estable para la identidad y las reglas de una entidad.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Definir una representación autoritativa mínima y estable para la identidad y las reglas de una entidad.	Cuando varias pantallas, índices o integraciones escriben su propia versión de un hecho, ninguna copia puede explicar cuál manda. Una corrección se convierte en una reconciliación manual.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Dato fechado · Reconstrucción total · Conjunto de datos reproducible

LABORATORIO DE CAMPO · 01

Objeto canónico

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Cuando varias pantallas, índices o integraciones escriben su propia versión de un hecho, ninguna copia puede explicar cuál manda. Una corrección se convierte en una reconciliación manual.

Ejercicio

Dibuje las rutas de escritura actuales. Marque cuál puede cambiar una regla de dominio y cierre las demás.

Resultado que debe conservar

Definir una representación autoritativa mínima y estable para la identidad y las reglas de una entidad.

Contrato mínimo

```
guardar(cambio) → validar → escribirCanónico → emitirRevisión
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida escrituras directas fuera de la frontera, entidades sin revisión y derivados sin origen. Se relaciona con Repositorio con ámbito, Proyección de lectura y Publicación por versión. El costo principal es diseñar migraciones y reglas de autoridad.

TARJETA DE REFERENCIA · 01

Objeto canónico

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar objeto canónico bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida escrituras directas fuera de la frontera, entidades sin revisión y derivados sin origen. Se relaciona con Repositorio con ámbito, Proyección de lectura y Publicación por versión. El costo principal es diseñar migraciones y reglas de autoridad.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

IMPLEMENTACIÓN COMPARADA · 01

Objeto canónico

Expediente de un árbol patrimonial

Varias brigadas guardan altura, especie y estado en archivos distintos.

Resultado: Un único expediente conserva identidad y reglas; los mapas son derivados.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Expediente de un árbol patrimonial
type Solicitud = Record<string, unknown>;

class ArbolPatrimonial {
  registrar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 1, evidencia: true, solicitud };
  }
}

const resultado = new ArbolPatrimonial().registrar({ arbolId:
"roble-42", especie: "Quercus costaricensis" });
```

Python

```
# Expediente de un árbol patrimonial
class ArbolPatrimonial:
    def registrar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 1, "evidencia": True, **solicitud}

resultado = ArbolPatrimonial().registrar({"detalle": "arbolId:
\"roble-42\", especie: \"Quercus costaricensis\""})
```

Java

```
// Expediente de un árbol patrimonial
record Solicitud(String detalle) {}

final class ArbolPatrimonial {
  Resultado registrar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(1, true, solicitud);
  }
}

var resultado = new ArbolPatrimonial().registrar(new
Solicitud("arbolId: \"roble-42\", especie: \"Quercus
costaricensis\""));
```

Go

```
// Expediente de un árbol patrimonial
type Solicitud struct { Detalle string }

func Registrar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 1, Evidencia: true}, nil
}

resultado, err := Registrar(Solicitud{Detalle: "arbolId:
\"roble-42\", especie: \"Quercus costaricensis\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

02 Dato fechado

Adaptación COBRIA | Evidencia: Contrato compuesto

La identidad, la validación y las versiones convierten entradas dispersas en documentos interpretables.

Propósito

Separar cuándo un hecho fue válido de cuándo el sistema lo conoció o registró.

Problema

Un único campo de fecha no permite reconstruir qué se sabía en un momento ni corregir eventos tardíos sin reescribir historia.

Analogía

Un periódico posee fecha del acontecimiento y fecha de publicación. Confundirlas cambia la historia narrada.

Solución y contrato

Cada observación declara tiempo de validez, tiempo de registro, revisión y procedencia. Las correcciones añaden conocimiento; no alteran silenciosamente el corte histórico.

Aplicación responsable

Úselo para eventos, tarifas, estados, auditoría y entrenamiento temporal. Puede omitirse cuando el tiempo carece de significado de dominio. Defina primero las preguntas temporales que deben responderse.

Métricas, costos y relaciones

Mida eventos tardíos, correcciones, cortes reproducibles y divergencia temporal. Se relaciona con Marca de avance temporal y Conjunto de datos reproducible. Aumenta almacenamiento y exige semántica clara.

MECÁNICA · MODELADO DOCUMENTAL

Cómo funciona Dato fechado

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
registrar(hecho, válidoEn, conocidoEn, procedencia)
```

> **Invariante principal.** Separar cuándo un hecho fue válido de cuándo el sistema lo conoció o registró.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Separar cuándo un hecho fue válido de cuándo el sistema lo conoció o registró.	Un único campo de fecha no permite reconstruir qué se sabía en un momento ni corregir eventos tardíos sin reescribir historia.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Repositorio con ámbito · Reconstrucción incremental · Característica versionada

LABORATORIO DE CAMPO · 02

Dato fechado

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Un único campo de fecha no permite reconstruir qué se sabía en un momento ni corregir eventos tardíos sin reescribir historia.

Ejercicio

Elija un hecho tardío y reconstruya qué se sabía antes y después de su registro.

Resultado que debe conservar

Separar cuándo un hecho fue válido de cuándo el sistema lo conoció o registró.

Contrato mínimo

```
registrar(hecho, válidoEn, conocidoEn, procedencia)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida eventos tardíos, correcciones, cortes reproducibles y divergencia temporal. Se relaciona con Marca de avance temporal y Conjunto de datos reproducible. Aumenta almacenamiento y exige semántica clara.

TARJETA DE REFERENCIA · 02

Dato fechado

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar dato fechado bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida eventos tardíos, correcciones, cortes reproducibles y divergencia temporal. Se relaciona con Marca de avance temporal y Conjunto de datos reproducible. Aumenta almacenamiento y exige semántica clara.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Dato fechado

Medición de un río después de una tormenta

Una lectura sin tiempo válido confunde cuándo ocurrió con cuándo se transmitió.

Resultado: La observación conserva tiempo observado, recibido y procedencia del sensor.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Medición de un río después de una tormenta
type Solicitud = Record<string, unknown>;

class MedicionRio {
  registrar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 2, evidencia: true, solicitud };
  }
}

const resultado = new MedicionRio().registrar({ rioId:
"terraba", nivelCm: 184, observadoEn: "2026-08-24T06:15:00Z" });
```

Python

```
# Medición de un río después de una tormenta
class MedicionRio:
    def registrar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 2, "evidencia": True, **solicitud}

resultado = MedicionRio().registrar({"detalle": "rioId:
\"terraba\", nivelCm: 184, observadoEn: \"2026-08-
24T06:15:00Z\""})
```

Java

```
// Medición de un río después de una tormenta
record Solicitud(String detalle) {}

final class MedicionRio {
  Resultado registrar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(2, true, solicitud);
  }
}

var resultado = new MedicionRio().registrar(new
Solicitud("rioId: \"terraba\", nivelCm: 184, observadoEn:
\"2026-08-24T06:15:00Z\""));
```

Go

```
// Medición de un río después de una tormenta
type Solicitud struct { Detalle string }

func Registrar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 2, Evidencia: true}, nil
}

resultado, err := Registrar(Solicitud{Detalle: "rioId:
\"terraba\", nivelCm: 184, observadoEn: \"2026-08-
24T06:15:00Z\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

03 Repositorio con ámbito

Patrón adaptado | Evidencia: Adaptación COBRIA

Antes: los ámbitos se mezclan. Después: el acceso decide el sendero antes de recuperar documentos.

Propósito

Encapsular persistencia y hacer obligatorio el límite organizacional en cada operación.

Problema

Un filtro de organización añadido al final puede olvidarse. Cachés, consultas auxiliares o tareas administrativas terminan mezclando datos.

Analogía

Es una caja de seguridad cuya llave abre un compartimento específico, no todo el edificio.

Solución y contrato

La interfaz no acepta operaciones sin ámbito. Identidad, claves, consultas, cachés y registros incluyen el mismo límite. La ausencia de ámbito falla antes de acceder al almacén.

Aplicación responsable

Úselo en sistemas multiinquilino o con fronteras de privacidad. No confíe únicamente en convenciones. Migre primero lecturas, luego escrituras y finalmente tareas de fondo.

Métricas, costos y relaciones

Mida solicitudes rechazadas, fugas cruzadas y claves de caché incompletas. Se relaciona con Contexto autorizado y Recuperación limitada por ámbito. Añade parámetros, índices y pruebas negativas.

MECÁNICA · SEGURIDAD POR ÁMBITO

Cómo funciona Repositorio con ámbito

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
obtener(ámbito, identidad) o fallarAntesDeLeer()
```

> **Invariante principal.** Encapsular persistencia y hacer obligatorio el límite organizacional en cada operación.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Encapsular persistencia y hacer obligatorio el límite organizacional en cada operación.	Un filtro de organización añadido al final puede olvidarse. Cachés, consultas auxiliares o tareas administrativas terminan mezclando datos.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Catálogo versionado · Publicación por versión · Recuperación limitada por ámbito

LABORATORIO DE CAMPO · 03

Repositorio con ámbito

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Un filtro de organización añadido al final puede olvidarse. Cachés, consultas auxiliares o tareas administrativas terminan mezclando datos.

Ejercicio

Escriba tres pruebas negativas: ámbito ausente, manipulado y válido con recurso ajeno.

Resultado que debe conservar

Encapsular persistencia y hacer obligatorio el límite organizacional en cada operación.

Contrato mínimo

```
obtener(ámbito, identidad) o fallarAntesDeLeer()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida solicitudes rechazadas, fugas cruzadas y claves de caché incompletas. Se relaciona con Contexto autorizado y Recuperación limitada por ámbito. Añade parámetros, índices y pruebas negativas.

TARJETA DE REFERENCIA · 03

Repositorio con ámbito

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar repositorio con ámbito bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida solicitudes rechazadas, fugas cruzadas y claves de caché incompletas. Se relaciona con Contexto autorizado y Recuperación limitada por ámbito. Añade parámetros, índices y pruebas negativas.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

IMPLEMENTACIÓN COMPARADA · 03

Repositorio con ámbito

Observaciones privadas de una reserva

Una consulta sin reservaId puede mezclar registros de áreas protegidas diferentes.

Resultado: El repositorio rechaza cualquier lectura que no declare primero la reserva.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Observaciones privadas de una reserva
type Solicitud = Record<string, unknown>;

class RepositorioObservaciones {
  buscar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 3, evidencia: true, solicitud };
  }
}

const resultado = new RepositorioObservaciones().buscar({
reservaId: "las-tablas", observacionId: "obs-184" });
```

Python

```
# Observaciones privadas de una reserva
class RepositorioObservaciones:
    def buscar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 3, "evidencia": True, **solicitud}

resultado = RepositorioObservaciones().buscar({"detalle":
"reservaId: \"las-tablas\", observacionId: \"obs-184\""})
```

Java

```
// Observaciones privadas de una reserva
record Solicitud(String detalle) {}

final class RepositorioObservaciones {
  Resultado buscar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(3, true, solicitud);
  }
}

var resultado = new RepositorioObservaciones().buscar(new
Solicitud("reservaId: \"las-tablas\", observacionId: \"obs-
184\""));
```

Go

```
// Observaciones privadas de una reserva
type Solicitud struct { Detalle string }

func Buscar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 3, Evidencia: true}, nil
}

resultado, err := Buscar(Solicitud{Detalle: "reservaId: \"las-
tablas\", observacionId: \"obs-184\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

04 Catálogo versionado

Contrato | Evidencia: Adaptación COBRIA

Las ediciones separadas permiten interpretar cada valor y aislar los desconocidos.

Propósito

Gobernar vocabularios de dominio sin incrustar significados cambiantes en código o datos derivados.

Problema

Un código como “A” cambia de significado, desaparece o llega desde una versión antigua; las proyecciones mezclan interpretaciones.

Analogía

Un diccionario posee edición. Conocer la palabra sin saber qué edición se utilizó puede producir una traducción equivocada.

Solución y contrato

El catálogo declara clave estable, versión, vigencia, sustituciones y política para valores desconocidos. Las transformaciones registran la edición consultada.

Aplicación responsable

Úselo cuando categorías afectan reglas, analítica o interoperabilidad. Evite convertir toda constante local en catálogo remoto. Defina cuarentena o migración para desconocidos.

Métricas, costos y relaciones

Mida valores desconocidos, versiones activas y derivados afectados. Se relaciona con Registro de dependencias y Conjunto reproducible. Su costo es coordinación y compatibilidad.

MECÁNICA · CATÁLOGOS Y VOCABULARIOS

Cómo funciona Catálogo versionado

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
resolver(clave, versión, fecha) o ponerEnCuarentena()
```

> **Invariante principal.** Gobernar vocabularios de dominio sin incrustar significados cambiantes en código o datos derivados.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Gobernar vocabularios de dominio sin incrustar significados cambiantes en código o datos derivados.	Un código como “A” cambia de significado, desaparece o llega desde una versión antigua; las proyecciones mezclan interpretaciones.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Proyección de lectura · Procesamiento idempotente · Herramienta de agente gobernada

LABORATORIO DE CAMPO · 04

Catálogo versionado

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Un código como “A” cambia de significado, desaparece o llega desde una versión antigua; las proyecciones mezclan interpretaciones.

Ejercicio

Tome un código ambiguo y defina versión, vigencia, sustitución y cuarentena.

Resultado que debe conservar

Gobernar vocabularios de dominio sin incrustar significados cambiantes en código o datos derivados.

Contrato mínimo

```
resolver(clave, versión, fecha) o ponerEnCuarentena()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida valores desconocidos, versiones activas y derivados afectados. Se relaciona con Registro de dependencias y Conjunto reproducible. Su costo es coordinación y compatibilidad.

TARJETA DE REFERENCIA · 04

Catálogo versionado

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar catálogo versionado bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida valores desconocidos, versiones activas y derivados afectados. Se relaciona con Registro de dependencias y Conjunto reproducible. Su costo es coordinación y compatibilidad.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

IMPLEMENTACIÓN COMPARADA · 04

Catálogo versionado

Códigos de especies usados durante diez años

El código AV-03 cambió de significado y los informes históricos quedan ambiguos.

Resultado: Cada observación se interpreta con la edición del catálogo que estaba vigente.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Códigos de especies usados durante diez años
type Solicitud = Record<string, unknown>;

class CatalogoEspecies {
  resolver(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 4, evidencia: true, solicitud };
  }
}

const resultado = new CatalogoEspecies().resolver({ codigo: "AV-
03", version: "2024.2" });
```

Python

```
# Códigos de especies usados durante diez años
class CatalogoEspecies:
    def resolver(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 4, "evidencia": True, **solicitud}

resultado = CatalogoEspecies().resolver({"detalle": "codigo:
\"AV-03\", version: \"2024.2\""})
```

Java

```
// Códigos de especies usados durante diez años
record Solicitud(String detalle) {}

final class CatalogoEspecies {
  Resultado resolver(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(4, true, solicitud);
  }
}

var resultado = new CatalogoEspecies().resolver(new
Solicitud("codigo: \"AV-03\", version: \"2024.2\""));
```

Go

```
// Códigos de especies usados durante diez años
type Solicitud struct { Detalle string }

func Resolver(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 4, Evidencia: true}, nil
}

resultado, err := Resolver(Solicitud{Detalle: "codigo: \"AV-
03\", version: \"2024.2\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

05 Proyección de lectura

Patrón adaptado | Evidencia: Establecido y adaptado

Los nidos sirven lecturas distintas sin desplazar la autoridad de las raíces.

Propósito

Mantener una representación derivada diseñada para una consulta concreta sin crear una nueva autoridad.

Problema

El modelo de escritura contiene más campos, relaciones o particiones de las que una consulta necesita. Leerlo directamente consume tiempo y bytes.

Analogía

Es el índice de un libro: acelera la búsqueda, pero puede rehacerse desde las páginas y no modifica la obra.

Solución y contrato

La ficha declara consulta beneficiada, fuente, transformación, versión, consistencia, verificador y reconstrucción. La aplicación la trata como descartable.

Aplicación responsable

Úsela después de medir una consulta y comparar un índice nativo. Evítela cuando la fuente ya responde dentro del objetivo o la frescura estricta no puede sostenerse.

Métricas, costos y relaciones

Mida p95, filas y bytes leídos, frescura, escrituras y almacenamiento. Se relaciona con Proyección de cobertura, Frontera lectura–escritura y Retiro de proyección.

MECÁNICA · PROYECCIONES DE LECTURA

Cómo funciona Proyección de lectura

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
proyectar(fuente, transformación, versión, ámbito)
```

> **Invariante principal.** Mantener una representación derivada diseñada para una consulta concreta sin crear una nueva autoridad.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Mantener una representación derivada diseñada para una consulta concreta sin crear una nueva autoridad.	El modelo de escritura contiene más campos, relaciones o particiones de las que una consulta necesita. Leerlo directamente consume tiempo y bytes.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Proyección de cobertura · Revisión obsoleta rechazada · Frontera lectura–escritura

LABORATORIO DE CAMPO · 05

Proyección de lectura

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

El modelo de escritura contiene más campos, relaciones o particiones de las que una consulta necesita. Leerlo directamente consume tiempo y bytes.

Ejercicio

Seleccione una consulta lenta y escriba exactamente filtros, orden, salida y frescura.

Resultado que debe conservar

Mantener una representación derivada diseñada para una consulta concreta sin crear una nueva autoridad.

Contrato mínimo

```
proyectar(fuente, transformación, versión, ámbito)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida p95, filas y bytes leídos, frescura, escrituras y almacenamiento. Se relaciona con Proyección de cobertura, Frontera lectura–escritura y Retiro de proyección.

TARJETA DE REFERENCIA · 05

Proyección de lectura

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar proyección de lectura bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida p95, filas y bytes leídos, frescura, escrituras y almacenamiento. Se relaciona con Proyección de cobertura, Frontera lectura–escritura y Retiro de proyección.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Proyección de lectura

Panel diario de avistamientos

La pantalla recorre miles de observaciones para contar especies cada vez que abre.

Resultado: Una vista reconstruible entrega conteos diarios sin reemplazar las observaciones.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Panel diario de avistamientos
type Solicitud = Record<string, unknown>;

class PanelAvistamientos {
  proyectar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 5, evidencia: true, solicitud };
  }
}

const resultado = new PanelAvistamientos().proyectar({
reservaId: "piedras-blancas", fecha: "2026-08-24" });
```

Python

```python
# Panel diario de avistamientos
class PanelAvistamientos:
    def proyectar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 5, "evidencia": True, **solicitud}

resultado = PanelAvistamientos().proyectar({"detalle":
"reservaId: \"piedras-blancas\", fecha: \"2026-08-24\""})
```

Java

```java
// Panel diario de avistamientos
record Solicitud(String detalle) {}

final class PanelAvistamientos {
  Resultado proyectar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(5, true, solicitud);
  }
}

var resultado = new PanelAvistamientos().proyectar(new
Solicitud("reservaId: \"piedras-blancas\", fecha: \"2026-08-
24\""));
```

Go

```go
// Panel diario de avistamientos
type Solicitud struct { Detalle string }

func Proyectar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 5, Evidencia: true}, nil
}

resultado, err := Proyectar(Solicitud{Detalle: "reservaId:
\"piedras-blancas\", fecha: \"2026-08-24\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

06 Proyección de cobertura

Mecanismo | Evidencia: Adaptación COBRIA

Los nidos sirven lecturas distintas sin desplazar la autoridad de las raíces.

Propósito

Responder una consulta utilizando una representación estrecha que contiene exactamente filtros, orden y salida.

Problema

Un índice encuentra filas, pero la consulta todavía debe visitar documentos o tablas anchas para completar la respuesta.

Analogía

Un formulario de viaje lleva solo pasaporte, boleto y dirección; no transporta todo el archivo personal.

Solución y contrato

La proyección incluye campos necesarios y procedencia mínima. Su índice cubre selección y orden. La equivalencia compara resultados lógicos con la fuente.

Aplicación responsable

Úsela cuando la recuperación de filas anchas domina el costo. Evítela si duplica datos sensibles o cambia con demasiada frecuencia. Compare siempre contra B1.

Métricas, costos y relaciones

Mida ahorro p95, bytes, penalización de escritura y umbral de equilibrio. El piloto del mecanismo mostró que la proyección no domina siempre al índice nativo; la réplica definitiva debe ejecutarse sobre motores NoSQL documentales.

MECÁNICA · PROYECCIONES DE LECTURA

Cómo funciona Proyección de cobertura

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
cubrir(filtros, orden, salida) sinCamposExtra()
```

> **Invariante principal.** Responder una consulta utilizando una representación estrecha que contiene exactamente filtros, orden y salida.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Responder una consulta utilizando una representación estrecha que contiene exactamente filtros, orden y salida.	Un índice encuentra filas, pero la consulta todavía debe visitar documentos o tablas anchas para completar la respuesta.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Reconstrucción total · Cola de salida transaccional · Índice antes que proyección

LABORATORIO DE CAMPO · 06

Proyección de cobertura

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Un índice encuentra filas, pero la consulta todavía debe visitar documentos o tablas anchas para completar la respuesta.

Ejercicio

Quite todos los campos no usados por la consulta y vuelva a medir bytes y costo.

Resultado que debe conservar

Responder una consulta utilizando una representación estrecha que contiene exactamente filtros, orden y salida.

Contrato mínimo

```
cubrir(filtros, orden, salida) sinCamposExtra()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida ahorro p95, bytes, penalización de escritura y umbral de equilibrio. El piloto del mecanismo mostró que la proyección no domina siempre al índice nativo; la réplica definitiva debe ejecutarse sobre motores NoSQL documentales.

TARJETA DE REFERENCIA · 06

Proyección de cobertura

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar proyección de cobertura bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida ahorro p95, bytes, penalización de escritura y umbral de equilibrio. El piloto del mecanismo mostró que la proyección no domina siempre al índice nativo; la réplica definitiva debe ejecutarse sobre motores NoSQL documentales.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Proyección de cobertura

Ficha breve para guardaparques sin conexión

La consulta móvil descarga expedientes completos cuando solo necesita ubicación y alerta.

Resultado: La proyección contiene exactamente los campos que necesita la ronda de campo.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Ficha breve para guardaparques sin conexión
type Solicitud = Record<string, unknown>;

class FichaGuardaparque {
  crear(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 6, evidencia: true, solicitud };
  }
}

const resultado = new FichaGuardaparque().crear({ senderoId:
"quebrada-azul", incluir: ["riesgo", "ubicacion"] });
```

Python

```
# Ficha breve para guardaparques sin conexión
class FichaGuardaparque:
    def crear(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 6, "evidencia": True, **solicitud}

resultado = FichaGuardaparque().crear({"detalle": "senderoId:
\"quebrada-azul\", incluir: [\"riesgo\", \"ubicacion\"]"})
```

Java

```
// Ficha breve para guardaparques sin conexión
record Solicitud(String detalle) {}

final class FichaGuardaparque {
  Resultado crear(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(6, true, solicitud);
  }
}

var resultado = new FichaGuardaparque().crear(new
Solicitud("senderoId: \"quebrada-azul\", incluir: [\"riesgo\",
\"ubicacion\"]"));
```

Go

```
// Ficha breve para guardaparques sin conexión
type Solicitud struct { Detalle string }

func Crear(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 6, Evidencia: true}, nil
}

resultado, err := Crear(Solicitud{Detalle: "senderoId:
\"quebrada-azul\", incluir: [\"riesgo\", \"ubicacion\"]"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

07 Reconstrucción total

Contrato | Evidencia: Contrato compuesto

La fuente permanece intacta mientras una versión derivada se reconstruye y verifica.

Propósito

Regenerar completamente una proyección desde fuentes y versiones declaradas.

Problema

Sin una reconstrucción probada, la pérdida o corrupción obliga a reparar manualmente una copia que ya no es realmente descartable.

Analogía

Es rehacer un índice desde el libro completo, no corregir entradas aisladas cuya procedencia se desconoce.

Solución y contrato

Se construye una versión candidata, se verifica conteo y contenido, y solo después se publica. La versión activa permanece visible durante el proceso.

Aplicación responsable

Impleméntela antes de optimizar la incremental. Ejecútela periódicamente y después de cambios de esquema. No publique una tabla parcialmente llena.

Métricas, costos y relaciones

Mida duración, exactitud, pico de almacenamiento y tiempo de cambio. Se relaciona con Publicación por versión y Reparación verificable.

MECÁNICA · RECONSTRUCCIÓN

Cómo funciona Reconstrucción total

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
candidata = reconstruir(canónicos); verificar(candidata); publicar()
```

> **Invariante principal.** Regenerar completamente una proyección desde fuentes y versiones declaradas.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Regenerar completamente una proyección desde fuentes y versiones declaradas.	Sin una reconstrucción probada, la pérdida o corrupción obliga a reparar manualmente una copia que ya no es realmente descartable.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Reconstrucción incremental · Reparación verificable · Retiro de proyección

LABORATORIO DE CAMPO · 07

Reconstrucción total

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Sin una reconstrucción probada, la pérdida o corrupción obliga a reparar manualmente una copia que ya no es realmente descartable.

Ejercicio

Elimine una copia en un entorno aislado y regenérela sin tocar la fuente.

Resultado que debe conservar

Regenerar completamente una proyección desde fuentes y versiones declaradas.

Contrato mínimo

```
candidata = reconstruir(canónicos); verificar(candidata); publicar()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida duración, exactitud, pico de almacenamiento y tiempo de cambio. Se relaciona con Publicación por versión y Reparación verificable.

TARJETA DE REFERENCIA · 07

Reconstrucción total

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar reconstrucción total bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida duración, exactitud, pico de almacenamiento y tiempo de cambio. Se relaciona con Publicación por versión y Reparación verificable.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Reconstrucción total

Mapa de calor de nacientes

Una nueva fórmula exige recalcular todo el mapa sin modificar mediciones originales.

Resultado: Se construye una versión candidata completa, se verifica y luego se publica.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Mapa de calor de nacientes
type Solicitud = Record<string, unknown>;

class ReconstruirMapaNacientes {
  ejecutar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 7, evidencia: true, solicitud };
  }
}

const resultado = new ReconstruirMapaNacientes().ejecutar({
cuencaId: "coto-brus", version: "v3" });
```

Python

```python
# Mapa de calor de nacientes
class ReconstruirMapaNacientes:
    def ejecutar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 7, "evidencia": True, **solicitud}

resultado = ReconstruirMapaNacientes().ejecutar({"detalle":
"cuencaId: \"coto-brus\", version: \"v3\""})
```

Java

```java
// Mapa de calor de nacientes
record Solicitud(String detalle) {}

final class ReconstruirMapaNacientes {
  Resultado ejecutar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(7, true, solicitud);
  }
}

var resultado = new ReconstruirMapaNacientes().ejecutar(new
Solicitud("cuencaId: \"coto-brus\", version: \"v3\""));
```

Go

```go
// Mapa de calor de nacientes
type Solicitud struct { Detalle string }

func Ejecutar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 7, Evidencia: true}, nil
}

resultado, err := Ejecutar(Solicitud{Detalle: "cuencaId: \"coto-
brus\", version: \"v3\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

08 Reconstrucción incremental

Mecanismo | Evidencia: Adaptación COBRIA

Una representación derivada vuelve a producirse y se compara con la fuente.

Propósito

Actualizar solo la fracción modificada manteniendo equivalencia con una reconstrucción total.

Problema

Rehacer todo el conjunto por un cambio pequeño desperdicia recursos y amplía la ventana de obsolescencia.

Analogía

Es actualizar las páginas cambiadas de un índice, conservando la posibilidad de regenerarlo completo si hay dudas.

Solución y contrato

Un registro de cambios identifica entidades y revisiones desde un punto de control. Cada lote es idempotente; una verificación periódica contrasta contra la ruta total.

Aplicación responsable

Úsela cuando $\Delta N \ll N$ y descubrir cambios cuesta menos que reconstruir. Evítela si no existe registro confiable o si la transformación depende globalmente de todos los registros.

Métricas, costos y relaciones

Mida $\Delta N/N$, duración, omisiones, duplicados y divergencia. Se relaciona con Punto de control, Idempotencia y Reconstrucción total.

MECÁNICA · RECONSTRUCCIÓN

Cómo funciona Reconstrucción incremental

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
desde(puntoDeControl).procesarUnaVezPorRevisión()
```

> **Invariante principal.** Actualizar solo la fracción modificada manteniendo equivalencia con una reconstrucción total.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Actualizar solo la fracción modificada manteniendo equivalencia con una reconstrucción total.	Rehacer todo el conjunto por un cambio pequeño desperdicia recursos y amplía la ventana de obsolescencia.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Publicación por versión · Conjunto de datos reproducible · Identidad documental estable

LABORATORIO DE CAMPO · 08

Reconstrucción incremental

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Rehacer todo el conjunto por un cambio pequeño desperdicia recursos y amplía la ventana de obsolescencia.

Ejercicio

Procese dos veces el mismo lote y compare el resultado con una reconstrucción total.

Resultado que debe conservar

Actualizar solo la fracción modificada manteniendo equivalencia con una reconstrucción total.

Contrato mínimo

```
desde(puntoDeControl).procesarUnaVezPorRevisión()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida Δ N/N , duración, omisiones, duplicados y divergencia. Se relaciona con Punto de control, Idempotencia y Reconstrucción total.

TARJETA DE REFERENCIA · 08

Reconstrucción incremental

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar reconstrucción incremental bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida Δ N/N , duración, omisiones, duplicados y divergencia. Se relaciona con Punto de control, Idempotencia y Reconstrucción total.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Reconstrucción incremental

Conteo de visitantes por sendero

Recalcular cinco años por una entrada nueva desperdicia tiempo y energía.

Resultado: Solo se procesa el cambio nuevo y se conserva un cursor verificable.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Conteo de visitantes por sendero
type Solicitud = Record<string, unknown>;

class ActualizarConteoSenderos {
  aplicar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 8, evidencia: true, solicitud };
  }
}

const resultado = new ActualizarConteoSenderos().aplicar({
eventoId: "paso-991", senderoId: "cotoncito" });
```

Python

```python
# Conteo de visitantes por sendero
class ActualizarConteoSenderos:
    def aplicar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 8, "evidencia": True, **solicitud}

resultado = ActualizarConteoSenderos().aplicar({"detalle":
"eventoId: \"paso-991\", senderoId: \"cotoncito\""})
```

Java

```java
// Conteo de visitantes por sendero
record Solicitud(String detalle) {}

final class ActualizarConteoSenderos {
  Resultado aplicar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(8, true, solicitud);
  }
}

var resultado = new ActualizarConteoSenderos().aplicar(new
Solicitud("eventoId: \"paso-991\", senderoId: \"cotoncito\""));
```

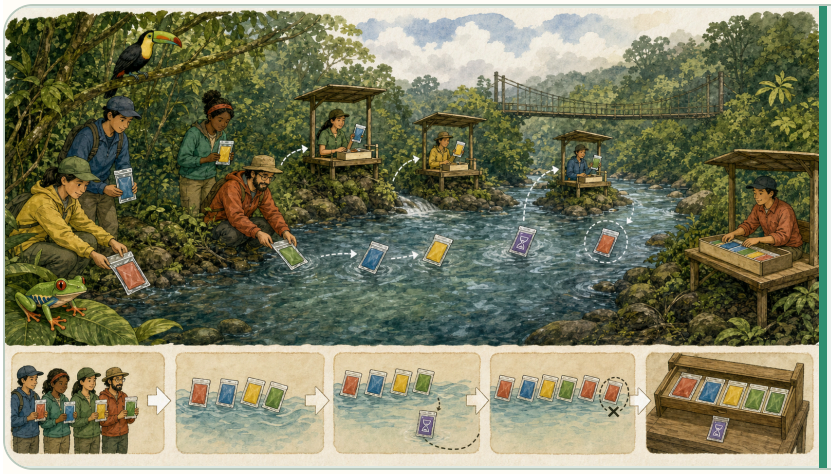
Go

```go
// Conteo de visitantes por sendero
type Solicitud struct { Detalle string }

func Aplicar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 8, Evidencia: true}, nil
}

resultado, err := Aplicar(Solicitud{Detalle: "eventoId: \"paso-
991\", senderoId: \"cotoncito\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

09 Publicación por versión

Mecanismo | Evidencia: Establecido y adaptado



Eventos repetidos, tardíos y ordenados atraviesan una ruta observable.

Propósito

Evitar que consumidores observen una proyección incompleta durante construcción o migración.

Problema

Vaciar y llenar la representación activa expone faltantes, mezclas de esquema o resultados transitorios.

Analogía

Un teatro ensaya detrás del telón y cambia el escenario cuando la nueva escena está lista.

Solución y contrato

Se construye una versión candidata aislada, se valida y se cambia un puntero o alias de forma atómica. La anterior se conserva durante una ventana reversible.

Aplicación responsable

Úsela para reconstrucciones largas o cambios incompatibles. Puede simplificarse si el motor ofrece actualización atómica equivalente. Nunca elimine la versión anterior antes de verificar.

Métricas, costos y relaciones

Mida pérdida visible, duración del cambio y almacenamiento máximo. Se relaciona con Reconstrucción total y Migración compatible.

Cómo funciona Publicación por versión

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
construir(vNueva); verificar(); cambiarAlias(); conservar(vAnterior)
```

> **Invariante principal.** Evitar que consumidores observen una proyección incompleta durante construcción o migración.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Evitar que consumidores observen una proyección incompleta durante construcción o migración.	Vaciar y llenar la representación activa expone faltantes, mezclas de esquema o resultados transitorios.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Procesamiento idempotente · Característica versionada · Validación en frontera

LABORATORIO DE CAMPO · 09

Publicación por versión

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Vaciar y llenar la representación activa expone faltantes, mezclas de esquema o resultados transitorios.

Ejercicio

Construya una candidata, interrumpa antes del cambio y compruebe que la activa sigue completa.

Resultado que debe conservar

Evitar que consumidores observen una proyección incompleta durante construcción o migración.

Contrato mínimo

```
construir(vNueva); verificar(); cambiarAlias(); conservar(vAnterior)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida pérdida visible, duración del cambio y almacenamiento máximo. Se relaciona con Reconstrucción total y Migración compatible.

TARJETA DE REFERENCIA · 09

Publicación por versión

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar publicación por versión bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida pérdida visible, duración del cambio y almacenamiento máximo. Se relaciona con Reconstrucción total y Migración compatible.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Publicación por versión

Nueva clasificación del estado del bosque

Los lectores podrían observar una mezcla de datos viejos y nuevos durante el cálculo.

Resultado: La versión candidata se activa mediante un único cambio de referencia.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Nueva clasificación del estado del bosque
type Solicitud = Record<string, unknown>;

class PublicadorCoberturaBosque {
  publicar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 9, evidencia: true, solicitud };
  }
}

const resultado = new PublicadorCoberturaBosque().publicar({
candidata: "cobertura-v7", esperada: "cobertura-v6" });
```

Python

```
# Nueva clasificación del estado del bosque
class PublicadorCoberturaBosque:
    def publicar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 9, "evidencia": True, **solicitud}

resultado = PublicadorCoberturaBosque().publicar({"detalle":
"candidata: \"cobertura-v7\", esperada: \"cobertura-v6\""})
```

Java

```
// Nueva clasificación del estado del bosque
record Solicitud(String detalle) {}

final class PublicadorCoberturaBosque {
  Resultado publicar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(9, true, solicitud);
  }
}

var resultado = new PublicadorCoberturaBosque().publicar(new
Solicitud("candidata: \"cobertura-v7\", esperada: \"cobertura-
v6\""));
```

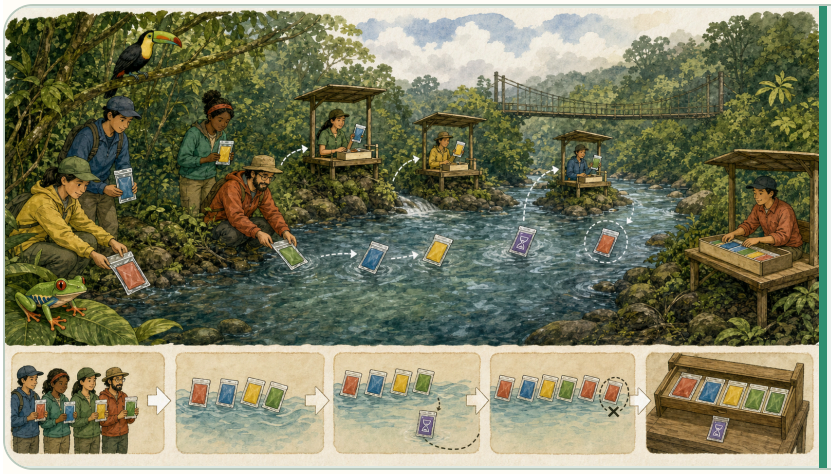
Go

```
// Nueva clasificación del estado del bosque
type Solicitud struct { Detalle string }

func Publicar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 9, Evidencia: true}, nil
}

resultado, err := Publicar(Solicitud{Detalle: "candidata:
\"cobertura-v7\", esperada: \"cobertura-v6\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

10 Procesamiento idempotente

Patrón de mensajería | Evidencia: Establecido



Eventos repetidos, tardíos y ordenados atraviesan una ruta observable.

Propósito

Garantizar que repetir la misma entrega no multiplique efectos.

Problema

Las redes reintentan y las colas pueden entregar más de una vez. Una operación no protegida duplica reservas, saldos o filas derivadas.

Analogía

Presionar dos veces el botón de un ascensor no debe crear dos ascensores.

Solución y contrato

Cada mensaje posee identidad estable. El consumidor registra procesamiento o utiliza una operación naturalmente idempotente dentro de la misma frontera de consistencia.

Aplicación responsable

Úselo en eventos, reintentos e importaciones. No confunda idempotencia con ignorar errores. Defina cuánto tiempo se conservan identificadores.

Métricas, costos y relaciones

Mida entregas duplicadas, efectos producidos y tamaño del registro. Se relaciona con Cola de salida y Reconstrucción incremental.

MECÁNICA · EVENTOS Y CONSISTENCIA

Cómo funciona Procesamiento idempotente

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
si procesado(idMensaje) entonces noHacerNada()
```

> **Invariante principal.** Garantizar que repetir la misma entrega no multiplique efectos.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Garantizar que repetir la misma entrega no multiplique efectos.	Las redes reintentan y las colas pueden entregar más de una vez. Una operación no protegida duplica reservas, saldos o filas derivadas.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Revisión obsoleta rechazada · Recuperación limitada por ámbito · Contrato documental versionado

LABORATORIO DE CAMPO · 10

Procesamiento idempotente

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Las redes reintentan y las colas pueden entregar más de una vez. Una operación no protegida duplica reservas, saldos o filas derivadas.

Ejercicio

Entregue dos veces el mismo mensaje y compruebe que solo existe un efecto.

Resultado que debe conservar

Garantizar que repetir la misma entrega no multiplique efectos.

Contrato mínimo

```
si procesado(idMensaje) entonces noHacerNada()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida entregas duplicadas, efectos producidos y tamaño del registro. Se relaciona con Cola de salida y Reconstrucción incremental.

TARJETA DE REFERENCIA · 10

Procesamiento idempotente

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar procesamiento idempotente bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida entregas duplicadas, efectos producidos y tamaño del registro. Se relaciona con Cola de salida y Reconstrucción incremental.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Procesamiento idempotente

Sensor que reenvía la misma lectura

La mala señal hace que una medición llegue tres veces y triplique el promedio.

Resultado: La clave del evento permite repetir la operación sin duplicar efectos.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Sensor que reenvía la misma lectura
type Solicitud = Record<string, unknown>;

class ProcesadorLecturas {
  procesar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 10, evidencia: true, solicitud };
  }
}

const resultado = new ProcesadorLecturas().procesar({ eventoId:
"sensor-17:8842", humedad: 86 });
```

Python

```python
# Sensor que reenvía la misma lectura
class ProcesadorLecturas:
    def procesar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 10, "evidencia": True, **solicitud}

resultado = ProcesadorLecturas().procesar({"detalle": "eventoId:
\"sensor-17:8842\", humedad: 86"})
```

Java

```java
// Sensor que reenvía la misma lectura
record Solicitud(String detalle) {}

final class ProcesadorLecturas {
  Resultado procesar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(10, true, solicitud);
  }
}

var resultado = new ProcesadorLecturas().procesar(new
Solicitud("eventoId: \"sensor-17:8842\", humedad: 86"));
```

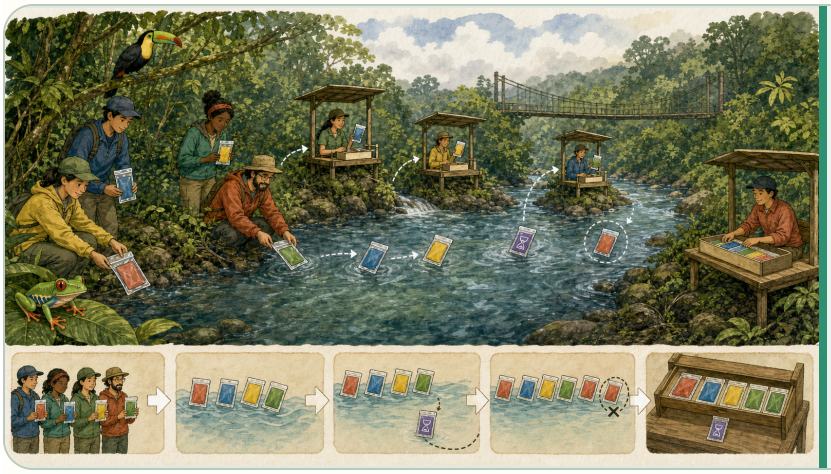
Go

```go
// Sensor que reenvía la misma lectura
type Solicitud struct { Detalle string }

func Procesar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 10, Evidencia: true}, nil
}

resultado, err := Procesar(Solicitud{Detalle: "eventoId:
\"sensor-17:8842\", humedad: 86"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

11 Revisión obsoleta rechazada

Control de concurrencia | Evidencia: Establecido y adaptado



Eventos repetidos, tardíos y ordenados atraviesan una ruta observable.

Propósito

Impedir que una actualización basada en estado antiguo sobrescriba conocimiento reciente.

Problema

Dos procesos leen la misma revisión y guardan cambios incompatibles; el último oculta al primero.

Analogía

Dos personas editan una hoja: la segunda debe saber que la primera ya entregó una nueva versión.

Solución y contrato

La escritura incluye la revisión esperada. Solo avanza si coincide con la actual; de lo contrario rechaza, recarga o aplica una política explícita de combinación.

Aplicación responsable

Úselo cuando las colisiones sean posibles y un bloqueo largo resulte caro. Evite reintentar automáticamente reglas no conmutativas.

Métricas, costos y relaciones

Mida rechazos, reintentos y conflictos resueltos. Se relaciona con Objeto canónico e Idempotencia.

MECÁNICA · EVENTOS Y CONSISTENCIA

Cómo funciona Revisión obsoleta rechazada

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
si revisiónEsperada ≠ revisiónActual entonces rechazar()
```

> **Invariante principal.** Impedir que una actualización basada en estado antiguo sobrescriba conocimiento reciente.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Impedir que una actualización basada en estado antiguo sobrescriba conocimiento reciente.	Dos procesos leen la misma revisión y guardan cambios incompatibles; el último oculta al primero.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Cola de salida transaccional · Herramienta de agente gobernada · Lectura coherente por revisión

LABORATORIO DE CAMPO · 11

Revisión obsoleta rechazada

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Dos procesos leen la misma revisión y guardan cambios incompatibles; el último oculta al primero.

Ejercicio

Envíe una revisión antigua y verifique rechazo, auditoría y ausencia de regresión.

Resultado que debe conservar

Impedir que una actualización basada en estado antiguo sobrescriba conocimiento reciente.

Contrato mínimo

```
si revisiónEsperada ≠ revisiónActual entonces rechazar()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida rechazos, reintentos y conflictos resueltos. Se relaciona con Objeto canónico e Idempotencia.

Revisión obsoleta rechazada

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar revisión obsoleta rechazada bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida rechazos, reintentos y conflictos resueltos. Se relaciona con Objeto canónico e Idempotencia.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Revisión obsoleta rechazada

Dos brigadas editan el mismo sendero

Una tableta sin conexión intenta guardar una descripción anterior sobre una revisión nueva.

Resultado: La escritura falla con conflicto y conserva la revisión más reciente.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Dos brigadas editan el mismo sendero
type Solicitud = Record<string, unknown>;

class EditorSendero {
  guardar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 11, evidencia: true, solicitud };
  }
}

const resultado = new EditorSendero().guardar({ senderoId:
"fila-cal", revisionEsperada: 12, dificultad: "alta" });
```

Python

```
# Dos brigadas editan el mismo sendero
class EditorSendero:
    def guardar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 11, "evidencia": True, **solicitud}

resultado = EditorSendero().guardar({"detalle": "senderoId:
\"fila-cal\", revisionEsperada: 12, dificultad: \"alta\""})
```

Java

```
// Dos brigadas editan el mismo sendero
record Solicitud(String detalle) {}

final class EditorSendero {
  Resultado guardar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(11, true, solicitud);
  }
}

var resultado = new EditorSendero().guardar(new
Solicitud("senderoId: \"fila-cal\", revisionEsperada: 12,
dificultad: \"alta\""));
```

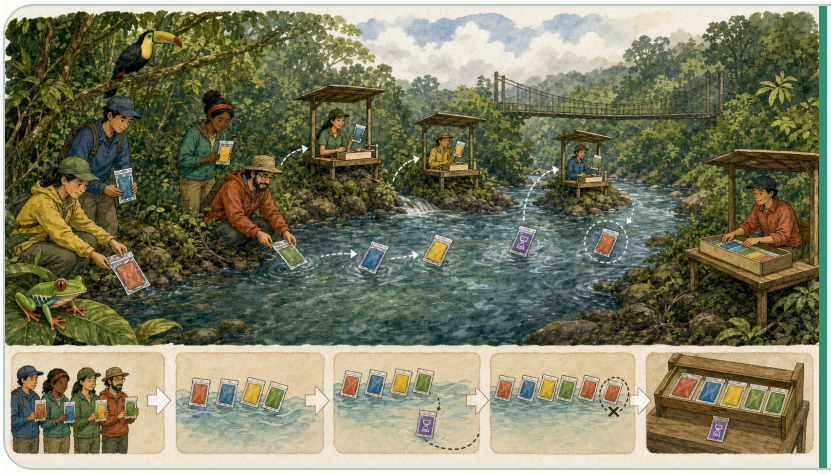
Go

```
// Dos brigadas editan el mismo sendero
type Solicitud struct { Detalle string }

func Guardar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 11, Evidencia: true}, nil
}

resultado, err := Guardar(Solicitud{Detalle: "senderoId: \"fila-
cal\", revisionEsperada: 12, dificultad: \"alta\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

12 Cola de salida transaccional

Patrón de integración | Evidencia: Establecido



Eventos repetidos, tardíos y ordenados atraviesan una ruta observable.

Propósito

Publicar cambios sin perder la relación entre escritura canónica y evento de integración.

Problema

Guardar datos y publicar un mensaje son dos operaciones; una puede tener éxito y la otra fallar.

Analogía

Es preparar la carta dentro de la misma oficina donde se registra el trámite y enviarla después con acuse.

Solución y contrato

La transacción escribe entidad y registro de salida. Un publicador reintentable entrega el mensaje y marca progreso. Los consumidores permanecen idempotentes.

Aplicación responsable

Úsela cuando el almacén no comparte transacción con el corredor. No la convierta en una cola sin retención, observabilidad o reparación.

Métricas, costos y relaciones

Mida retraso, reintentos, mensajes atascados y duplicados. Se relaciona con Proyección asíncrona y Punto de control.

MECÁNICA · EVENTOS Y CONSISTENCIA

Cómo funciona Cola de salida transaccional

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
enUnaOperación { guardarCanónico(); guardarEventoSalida(); }
```

> **Invariante principal.** Publicar cambios sin perder la relación entre escritura canónica y evento de integración.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Publicar cambios sin perder la relación entre escritura canónica y evento de integración.	Guardar datos y publicar un mensaje son dos operaciones; una puede tener éxito y la otra fallar.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Reparación verificable · Frontera lectura–escritura · Ventana de frescura declarada

LABORATORIO DE CAMPO · 12

Cola de salida transaccional

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Guardar datos y publicar un mensaje son dos operaciones; una puede tener éxito y la otra fallar.

Ejercicio

Interrumpa el publicador después de guardar la entidad y confirme que el evento sigue recuperable.

Resultado que debe conservar

Publicar cambios sin perder la relación entre escritura canónica y evento de integración.

Contrato mínimo

```
enUnaOperación { guardarCanónico(); guardarEventoSalida(); }
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida retraso, reintentos, mensajes atascados y duplicados. Se relaciona con Proyección asíncrona y Punto de control.

TARJETA DE REFERENCIA · 12

Cola de salida transaccional

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar cola de salida transaccional bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida retraso, reintentos, mensajes atascados y duplicados. Se relaciona con Proyección asíncrona y Punto de control.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Cola de salida transaccional

Alerta por deslizamiento junto con la medición

Guardar la alerta después de la medición puede fallar y dejar el riesgo sin notificación.

Resultado: Documento y mensaje pendiente nacen juntos; un trabajador publica después.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Alerta por deslizamiento junto con la medición
type Solicitud = Record<string, unknown>;

class RegistroConAlerta {
  guardar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 12, evidencia: true, solicitud };
  }
}

const resultado = new RegistroConAlerta().guardar({ sensorId:
"ladera-8", riesgo: "alto" });
```

Python

```python
# Alerta por deslizamiento junto con la medición
class RegistroConAlerta:
    def guardar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 12, "evidencia": True, **solicitud}

resultado = RegistroConAlerta().guardar({"detalle": "sensorId:
\"ladera-8\", riesgo: \"alto\""})
```

Java

```java
// Alerta por deslizamiento junto con la medición
record Solicitud(String detalle) {}

final class RegistroConAlerta {
  Resultado guardar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(12, true, solicitud);
  }
}

var resultado = new RegistroConAlerta().guardar(new
Solicitud("sensorId: \"ladera-8\", riesgo: \"alto\""));
```

Go

```go
// Alerta por deslizamiento junto con la medición
type Solicitud struct { Detalle string }

func Guardar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 12, Evidencia: true}, nil
}

resultado, err := Guardar(Solicitud{Detalle: "sensorId:
\"ladera-8\", riesgo: \"alto\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

13 Reparación verificable

Mecanismo | Evidencia: Contrato compuesto

Una representación derivada vuelve a producirse y se compara con la fuente.

Propósito

Detectar diferencias entre fuente y proyección y recuperar equivalencia sin edición manual opaca.

Problema

Una proyección puede corromperse aunque la actualización habitual parezca correcta. Sin verificador, el defecto se descubre mediante usuarios.

Analogía

Es comparar inventario físico con registro y reconstruir el estante, no ajustar el número para ocultar la diferencia.

Solución y contrato

El verificador calcula faltantes, extras y valores distintos. La reparación genera una nueva versión desde la fuente y conserva evidencia del incidente.

Aplicación responsable

Úsela como prueba periódica y procedimiento de incidente. Evite modificar ambos lados para hacer coincidir cifras. La fuente debe estar definida antes de reparar.

Métricas, costos y relaciones

Mida exactitud, tiempo de detección y recuperación. Se relaciona con Reconstrucción total, Publicación por versión y Registro de procedencia.

MECÁNICA · RECONSTRUCCIÓN

Cómo funciona Reparación verificable

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
si !equivalente(derivado, esperado) entonces reconstruirYVerificar()
```

> **Invariante principal.** Detectar diferencias entre fuente y proyección y recuperar equivalencia sin edición manual opaca.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Detectar diferencias entre fuente y proyección y recuperar equivalencia sin edición manual opaca.	Una proyección puede corromperse aunque la actualización habitual parezca correcta. Sin verificador, el defecto se descubre mediante usuarios.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Conjunto de datos reproducible · Índice antes que proyección · Manifiesto de proyección

LABORATORIO DE CAMPO · 13

Reparación verificable

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Una proyección puede corromperse aunque la actualización habitual parezca correcta. Sin verificador, el defecto se descubre mediante usuarios.

Ejercicio

Corrompa una muestra derivada, detecte la divergencia y repare desde canónicos.

Resultado que debe conservar

Detectar diferencias entre fuente y proyección y recuperar equivalencia sin edición manual opaca.

Contrato mínimo

```
si !equivalente(derivado, esperado) entonces reconstruirYVerificar()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida exactitud, tiempo de detección y recuperación. Se relaciona con Reconstrucción total, Publicación por versión y Registro de procedencia.

Reparación verificable

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar reparación verificable bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida exactitud, tiempo de detección y recuperación. Se relaciona con Reconstrucción total, Publicación por versión y Registro de procedencia.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Reparación verificable

Índice de especies incompleto

Una interrupción dejó veinte observaciones fuera del índice de búsqueda.

Resultado: La reparación compara fuente e índice, corrige diferencias y vuelve a medir.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Índice de especies incompleto
type Solicitud = Record<string, unknown>;

class ReparadorIndiceEspecies {
  reparar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 13, evidencia: true, solicitud };
  }
}

const resultado = new ReparadorIndiceEspecies().reparar({
reservaId: "golfo-dulce", lote: 100 });
```

Python

```python
# Índice de especies incompleto
class ReparadorIndiceEspecies:
    def reparar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 13, "evidencia": True, **solicitud}

resultado = ReparadorIndiceEspecies().reparar({"detalle":
"reservaId: \"golfo-dulce\", lote: 100"})
```

Java

```java
// Índice de especies incompleto
record Solicitud(String detalle) {}

final class ReparadorIndiceEspecies {
  Resultado reparar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(13, true, solicitud);
  }
}

var resultado = new ReparadorIndiceEspecies().reparar(new
Solicitud("reservaId: \"golfo-dulce\", lote: 100"));
```

Go

```go
// Índice de especies incompleto
type Solicitud struct { Detalle string }

func Reparar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 13, Evidencia: true}, nil
}

resultado, err := Reparar(Solicitud{Detalle: "reservaId:
\"golfo-dulce\", lote: 100"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

14 Conjunto de datos reproducible

Contrato analítico | Evidencia: Adaptación COBRIA

Particiones, transformaciones y resultados mantienen un linaje visible.

Propósito

Poder regenerar un conjunto analítico con la misma selección, corte, vocabulario y transformación.

Problema

Un archivo exportado sin contexto no explica qué registros faltan, qué versión utilizó ni cómo repetir un resultado.

Analogía

Es una receta con ingredientes, cantidades y fecha de cosecha, no solo una fotografía del plato.

Solución y contrato

El contrato registra fuentes, ámbitos autorizados, tiempo de corte, consulta, transformación, esquema, catálogo, resumen criptográfico y política de faltantes.

Aplicación responsable

Úselo para minería, entrenamiento, auditoría e informes. No publique datos personales por reproducibilidad; separe metadatos repetibles de contenido restringido.

Métricas, costos y relaciones

Mida cobertura de procedencia, diferencias entre reconstrucciones y registros en cuarentena. Se relaciona con Dato fechado y Característica versionada.

MECÁNICA · ANALÍTICA REPRODUCIBLE

Cómo funciona Conjunto de datos reproducible

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
conjunto = construir(corte, fuentes, versiones, semilla)
```

> **Invariante principal.** Poder regenerar un conjunto analítico con la misma selección, corte, vocabulario y transformación.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Poder regenerar un conjunto analítico con la misma selección, corte, vocabulario y transformación.	Un archivo exportado sin contexto no explica qué registros faltan, qué versión utilizó ni cómo repetir un resultado.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Característica versionada · Retiro de proyección · Partición analítica sin fuga

LABORATORIO DE CAMPO · 14

Conjunto de datos reproducible

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Un archivo exportado sin contexto no explica qué registros faltan, qué versión utilizó ni cómo repetir un resultado.

Ejercicio

Reconstruya el mismo corte dos veces y compare manifiesto, conteos y huellas.

Resultado que debe conservar

Poder regenerar un conjunto analítico con la misma selección, corte, vocabulario y transformación.

Contrato mínimo

```
conjunto = construir(corte, fuentes, versiones, semilla)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida cobertura de procedencia, diferencias entre reconstrucciones y registros en cuarentena. Se relaciona con Dato fechado y Característica versionada.

Conjunto de datos reproducible

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar conjunto de datos reproducible bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida cobertura de procedencia, diferencias entre reconstrucciones y registros en cuarentena. Se relaciona con Dato fechado y Característica versionada.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Conjunto de datos reproducible

Estudio anual de presencia de aves

El conjunto cambia cada vez que alguien corrige una observación histórica.

Resultado: El manifiesto fija consulta, corte, catálogo, transformación y huellas.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Estudio anual de presencia de aves
type Solicitud = Record<string, unknown>;

class ConstructorDatasetAves {
  construir(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 14, evidencia: true, solicitud };
  }
}

const resultado = new ConstructorDatasetAves().construir({
corte: "2026-07-31", catalogo: "aves-5", semilla: 420 });
```

Python

```
# Estudio anual de presencia de aves
class ConstructorDatasetAves:
    def construir(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 14, "evidencia": True, **solicitud}

resultado = ConstructorDatasetAves().construir({"detalle":
"corte: \"2026-07-31\", catalogo: \"aves-5\", semilla: 420"})
```

Java

```
// Estudio anual de presencia de aves
record Solicitud(String detalle) {}

final class ConstructorDatasetAves {
  Resultado construir(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(14, true, solicitud);
  }
}

var resultado = new ConstructorDatasetAves().construir(new
Solicitud("corte: \"2026-07-31\", catalogo: \"aves-5\", semilla:
420"));
```

Go

```
// Estudio anual de presencia de aves
type Solicitud struct { Detalle string }

func Construir(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 14, Evidencia: true}, nil
}

resultado, err := Construir(Solicitud{Detalle: "corte: \"2026-
07-31\", catalogo: \"aves-5\", semilla: 420"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

15 Característica versionada

Contrato de IA | Evidencia: Establecido y adaptado

Particiones, transformaciones y resultados mantienen un linaje visible.

Propósito

Definir una variable de modelo como transformación gobernada, no como columna accidental.

Problema

Entrenamiento e inferencia calculan valores similares con código, ventanas o catálogos diferentes.

Analogía

Es una unidad de medida calibrada: el número carece de sentido sin definición y versión.

Solución y contrato

La ficha declara entidad, fórmula, ventana, fuente, versión, valores ausentes, frescura y equivalencia fuera de línea–en línea.

Aplicación responsable

Úsela cuando una variable se comparte o influye en decisiones. Evite crear un almacén complejo para experimentos desechables sin ruta de producción.

Métricas, costos y relaciones

Mida divergencia, frescura, uso y deriva. Se relaciona con Conjunto reproducible y Registro de dependencias.

MECÁNICA · ANALÍTICA REPRODUCIBLE

Cómo funciona Característica versionada

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
característica = calcular(definición, tiempoEvento, versión)
```

> **Invariante principal.** Definir una variable de modelo como transformación gobernada, no como columna accidental.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Definir una variable de modelo como transformación gobernada, no como columna accidental.	Entrenamiento e inferencia calculan valores similares con código, ventanas o catálogos diferentes.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Recuperación limitada por ámbito · Identidad documental estable · Abstención con evidencia

LABORATORIO DE CAMPO · 15

Característica versionada

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Entrenamiento e inferencia calculan valores similares con código, ventanas o catálogos diferentes.

Ejercicio

Cambie una definición de variable y confirme que produce una versión nueva.

Resultado que debe conservar

Definir una variable de modelo como transformación gobernada, no como columna accidental.

Contrato mínimo

```
característica = calcular(definición, tiempoEvento, versión)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida divergencia, frescura, uso y deriva. Se relaciona con Conjunto reproducible y Registro de dependencias.

Característica versionada

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar característica versionada bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida divergencia, frescura, uso y deriva. Se relaciona con Conjunto reproducible y Registro de dependencias.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Característica versionada

Índice de sequedad para riesgo de incendio

Cambiar la fórmula altera predicciones sin explicar por qué.

Resultado: El valor viaja con la versión exacta de la fórmula que lo produjo.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Índice de sequedad para riesgo de incendio
type Solicitud = Record<string, unknown>;

class IndiceSequedad {
  calcular(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 15, evidencia: true, solicitud };
  }
}

const resultado = new IndiceSequedad().calcular({ humedad: 31,
lluvia7d: 4, formula: "is-2" });
```

Python

```
# Índice de sequedad para riesgo de incendio
class IndiceSequedad:
    def calcular(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 15, "evidencia": True, **solicitud}

resultado = IndiceSequedad().calcular({"detalle": "humedad: 31,
lluvia7d: 4, formula: \"is-2\""})
```

Java

```
// Índice de sequedad para riesgo de incendio
record Solicitud(String detalle) {}

final class IndiceSequedad {
  Resultado calcular(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(15, true, solicitud);
  }
}

var resultado = new IndiceSequedad().calcular(new
Solicitud("humedad: 31, lluvia7d: 4, formula: \"is-2\""));
```

Go

```
// Índice de sequedad para riesgo de incendio
type Solicitud struct { Detalle string }

func Calcular(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 15, Evidencia: true}, nil
}

resultado, err := Calcular(Solicitud{Detalle: "humedad: 31,
lluvia7d: 4, formula: \"is-2\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

16 Recuperación limitada por ámbito

Control de IA | Evidencia: Contrato compuesto

La evidencia conserva un hilo visible desde el documento hasta la recuperación autorizada.

Propósito

Impedir que búsqueda semántica o recuperación aumentada crucen fronteras de autorización.

Problema

Ordenar primero y filtrar después puede recuperar fragmentos prohibidos, contaminar contexto o revelar existencia.

Analogía

Una bibliotecaria entra primero a la sala autorizada y después busca el libro; no revisa todas las salas y oculta resultados al final.

Solución y contrato

El ámbito y la política restringen candidatos antes de similitud. Cada fragmento conserva fuente, versión y autorización; la respuesta enlaza evidencia o se abstiene.

Aplicación responsable

Úsela en buscadores multiinquilino y asistentes. No trate metadatos de autorización como texto opcional. Pruebe consultas negativas e inyección.

Métricas, costos y relaciones

Mida precisión, exhaustividad, fugas, abstención y afirmaciones sin fuente. Se relaciona con Repositorio con ámbito y Fragmento con evidencia.

MECÁNICA · IA CON PROCEDENCIA

Cómo funciona Recuperación limitada por ámbito

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
candidatos = buscar(dentroDe(ámbito, política), consulta)
```

> **Invariante principal.** Impedir que búsqueda semántica o recuperación aumentada crucen fronteras de autorización.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Impedir que búsqueda semántica o recuperación aumentada crucen fronteras de autorización.	Ordenar primero y filtrar después puede recuperar fragmentos prohibidos, contaminar contexto o revelar existencia.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Herramienta de agente gobernada · Validación en frontera · Presupuesto de derivados

LABORATORIO DE CAMPO · 16

Recuperación limitada por ámbito

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Ordenar primero y filtrar después puede recuperar fragmentos prohibidos, contaminar contexto o revelar existencia.

Ejercicio

Intente recuperar un fragmento de otro ámbito y compruebe que nunca fue candidato.

Resultado que debe conservar

Impedir que búsqueda semántica o recuperación aumentada crucen fronteras de autorización.

Contrato mínimo

```
candidatos = buscar(dentroDe(ámbito, política), consulta)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida precisión, exhaustividad, fugas, abstención y afirmaciones sin fuente. Se relaciona con Repositorio con ámbito y Fragmento con evidencia.

Recuperación limitada por ámbito

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar recuperación limitada por ámbito bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida precisión, exhaustividad, fugas, abstención y afirmaciones sin fuente. Se relaciona con Repositorio con ámbito y Fragmento con evidencia.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Recuperación limitada por ámbito

Asistente sobre una estación biológica

El asistente podría citar documentos pertenecientes a otra estación.

Resultado: Filtro, búsqueda y citas conservan el mismo ámbito autorizado.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Asistente sobre una estación biológica
type Solicitud = Record<string, unknown>;

class BuscadorEstacion {
  recuperar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 16, evidencia: true, solicitud };
  }
}

const resultado = new BuscadorEstacion().recuperar({ estacionId:
"las-cruces", pregunta: "¿Qué floreció esta semana?" });
```

Python

```python
# Asistente sobre una estación biológica
class BuscadorEstacion:
    def recuperar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 16, "evidencia": True, **solicitud}

resultado = BuscadorEstacion().recuperar({"detalle":
"estacionId: \"las-cruces\", pregunta: \"¿Qué floreció esta
semana?\""})
```

Java

```java
// Asistente sobre una estación biológica
record Solicitud(String detalle) {}

final class BuscadorEstacion {
  Resultado recuperar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(16, true, solicitud);
  }
}

var resultado = new BuscadorEstacion().recuperar(new
Solicitud("estacionId: \"las-cruces\", pregunta: \"¿Qué floreció
esta semana?\""));
```

Go

```go
// Asistente sobre una estación biológica
type Solicitud struct { Detalle string }

func Recuperar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 16, Evidencia: true}, nil
}

resultado, err := Recuperar(Solicitud{Detalle: "estacionId:
\"las-cruces\", pregunta: \"¿Qué floreció esta semana?\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

17 Herramienta de agente gobernada

Control de IA | Evidencia: Experimental

La respuesta enlaza evidencia autorizada o se abstiene cuando no la posee.

Propósito

Permitir que un agente ejecute casos de uso sin saltarse autorización, validación o confirmación.

Problema

Un modelo que llama directamente bases o servicios internos convierte una inferencia incierta en efecto real.

Analogía

Es un aprendiz con herramientas etiquetadas: puede observar libremente, pero necesita supervisión para acciones irreversibles.

Solución y contrato

Cada herramienta declara entrada, ámbito, riesgo, idempotencia, permisos, evidencia y confirmación. El agente invoca casos de uso; no escribe proyecciones ni fuentes directamente.

Aplicación responsable

Úsela solo con límites, registros y capacidad de revocación. Evite autonomía de alto riesgo basada únicamente en una instrucción textual.

Métricas, costos y relaciones

Mida acciones rechazadas, confirmaciones, efectos duplicados y decisiones sin evidencia. Se relaciona con Contexto autorizado e Idempotencia.

MECÁNICA · IA CON PROCEDENCIA

Cómo funciona Herramienta de agente gobernada

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
herramienta.validar(); exigirCapacidad(); confirmarSiRiesgo(); auditar()
```

> **Invariante principal.** Permitir que un agente ejecute casos de uso sin saltarse autorización, validación o confirmación.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Permitir que un agente ejecute casos de uso sin saltarse autorización, validación o confirmación.	Un modelo que llama directamente bases o servicios internos convierte una inferencia incierta en efecto real.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Frontera lectura–escritura · Contrato documental versionado · Auditoría narrativa reconstruible

LABORATORIO DE CAMPO · 17

Herramienta de agente gobernada

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Un modelo que llama directamente bases o servicios internos convierte una inferencia incierta en efecto real.

Ejercicio

Repita una acción de alto riesgo sin confirmación y exija abstención auditable.

Resultado que debe conservar

Permitir que un agente ejecute casos de uso sin saltarse autorización, validación o confirmación.

Contrato mínimo

```
herramienta.validar(); exigirCapacidad(); confirmarSiRiesgo(); auditar()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida acciones rechazadas, confirmaciones, efectos duplicados y decisiones sin evidencia. Se relaciona con Contexto autorizado e Idempotencia.

Herramienta de agente gobernada

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar herramienta de agente gobernada bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida acciones rechazadas, confirmaciones, efectos duplicados y decisiones sin evidencia. Se relaciona con Contexto autorizado e Idempotencia.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Herramienta de agente gobernada

Asistente que propone cerrar un sendero

Un modelo no debe cerrar rutas por sí mismo al detectar lluvia intensa.

Resultado: La herramienta valida permiso, muestra evidencia y exige aprobación humana.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Asistente que propone cerrar un sendero
type Solicitud = Record<string, unknown>;

class AgenteSenderos {
  proponer(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 17, evidencia: true, solicitud };
  }
}

const resultado = new AgenteSenderos().proponer({ senderoId:
"cerro-pittier", accion: "cerrar" });
```

Python

```python
# Asistente que propone cerrar un sendero
class AgenteSenderos:
    def proponer(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 17, "evidencia": True, **solicitud}

resultado = AgenteSenderos().proponer({"detalle": "senderoId:
\"cerro-pittier\", accion: \"cerrar\""})
```

Java

```java
// Asistente que propone cerrar un sendero
record Solicitud(String detalle) {}

final class AgenteSenderos {
  Resultado proponer(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(17, true, solicitud);
  }
}

var resultado = new AgenteSenderos().proponer(new
Solicitud("senderoId: \"cerro-pittier\", accion: \"cerrar\""));
```

Go

```go
// Asistente que propone cerrar un sendero
type Solicitud struct { Detalle string }

func Proponer(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 17, Evidencia: true}, nil
}

resultado, err := Proponer(Solicitud{Detalle: "senderoId:
\"cerro-pittier\", accion: \"cerrar\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

18 Frontera lectura–escritura

Regla de decisión | Evidencia: Experimental con evidencia

La arquitectura conserva rutas útiles y retira las que ya no justifican su costo.

Propósito

Decidir cuantitativamente si el ahorro de lectura compensa mantenimiento adicional.

Problema

Las proyecciones suelen adoptarse por intuición y permanecen aun cuando un índice nativo sería suficiente.

Analogía

Una carretera nueva se justifica solo si el ahorro acumulado de viajes paga construcción y mantenimiento.

Solución y contrato

Si $\Delta Q=Q_{B1}-Q_{C1}>0$ y $\Delta W=W_{C1}-W_{B1}$, C1 compensa temporalmente cuando $Q/W>\Delta W/\Delta Q$. Se añaden almacenamiento, personal y riesgo cuando estén disponibles.

Aplicación responsable

Mida en el entorno objetivo y por percentil. Si $\Delta Q\leq 0$, B1 domina en esa métrica. No generalice el umbral entre motores, escalas o consultas.

Métricas, costos y relaciones

El piloto observó umbrales entre 1.49 y 38.87 lecturas por escritura; son evidencia del método, no parámetros transferibles a NoSQL. Se relaciona con Proyección de cobertura y Retiro de proyección.

MECÁNICA · GOBERNANZA DEL CICLO DE VIDA

Cómo funciona Frontera lectura–escritura

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
adoptar si ahorroLectura > costoEscritura + costoOperacional
```

> **Invariante principal.** Decidir cuantitativamente si el ahorro de lectura compensa mantenimiento adicional.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Decidir cuantitativamente si el ahorro de lectura compensa mantenimiento adicional.	Las proyecciones suelen adoptarse por intuición y permanecen aun cuando un índice nativo sería suficiente.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Índice antes que proyección · Lectura coherente por revisión · Objeto canónico

LABORATORIO DE CAMPO · 18

Frontera lectura–escritura

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Las proyecciones suelen adoptarse por intuición y permanecen aun cuando un índice nativo sería suficiente.

Ejercicio

Calcule el punto de equilibrio con costos medidos y repita con el doble de escrituras.

Resultado que debe conservar

Decidir cuantitativamente si el ahorro de lectura compensa mantenimiento adicional.

Contrato mínimo

```
adoptar si ahorroLectura > costoEscritura + costoOperacional
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

El piloto observó umbrales entre 1.49 y 38.87 lecturas por escritura; son evidencia del método, no parámetros transferibles a NoSQL. Se relaciona con Proyección de cobertura y Retiro de proyección.

TARJETA DE REFERENCIA · 18

Frontera lectura–escritura

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar frontera lectura–escritura bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

El piloto observó umbrales entre 1.49 y 38.87 lecturas por escritura; son evidencia del método, no parámetros transferibles a NoSQL. Se relaciona con Proyección de cobertura y Retiro de proyección.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Frontera lectura–escritura

Resumen público de calidad del agua

Editar el resumen directamente lo convertiría en una segunda autoridad.

Resultado: El resumen solo se lee; toda corrección entra por la medición canónica.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Resumen público de calidad del agua
type Solicitud = Record<string, unknown>;

class ResumenCalidadAgua {
  consultar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 18, evidencia: true, solicitud };
  }
}

const resultado = new ResumenCalidadAgua().consultar({ cuencaId:
"sixaola", mes: "2026-08" });
```

Python

```
# Resumen público de calidad del agua
class ResumenCalidadAgua:
    def consultar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 18, "evidencia": True, **solicitud}

resultado = ResumenCalidadAgua().consultar({"detalle":
"cuencaId: \"sixaola\", mes: \"2026-08\""})
```

Java

```
// Resumen público de calidad del agua
record Solicitud(String detalle) {}

final class ResumenCalidadAgua {
  Resultado consultar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(18, true, solicitud);
  }
}

var resultado = new ResumenCalidadAgua().consultar(new
Solicitud("cuencaId: \"sixaola\", mes: \"2026-08\""));
```

Go

```
// Resumen público de calidad del agua
type Solicitud struct { Detalle string }

func Consultar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 18, Evidencia: true}, nil
}

resultado, err := Consultar(Solicitud{Detalle: "cuencaId:
\"sixaola\", mes: \"2026-08\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

19 Índice antes que proyección

Heurística | Evidencia: Adaptación COBRIA

La arquitectura conserva rutas útiles y retira las que ya no justifican su costo.

Propósito

Exigir una línea base nativa competente antes de añadir una representación derivada.

Problema

Comparar solo contra escaneo exagera el beneficio y oculta una solución más sencilla del motor.

Analogía

Antes de construir una vía paralela, se mejora la señalización de la vía existente.

Solución y contrato

B0 diagnostica el costo bruto; B1 usa el mejor índice razonable; C1 debe compararse con ambos y pagar su mantenimiento. La equivalencia lógica de respuestas es obligatoria.

Aplicación responsable

Úsela en toda decisión de materialización. Excepciones requieren documentar por qué el motor no puede indexar la consulta.

Métricas, costos y relaciones

Mida plan, filas, bytes, p95 y escritura. En la fase confirmatoria, C1 solo superó significativamente a B1 en cuatro de nueve celdas.

MECÁNICA · GOBERNANZA DEL CICLO DE VIDA

Cómo funciona Índice antes que proyección

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
medir(consultaConÍndice) antesDe crearProyección()
```

> **Invariante principal.** Exigir una línea base nativa competente antes de añadir una representación derivada.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Exigir una línea base nativa competente antes de añadir una representación derivada.	Comparar solo contra escaneo exagera el beneficio y oculta una solución más sencilla del motor.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Retiro de proyección · Ventana de frescura declarada · Dato fechado

LABORATORIO DE CAMPO · 19

Índice antes que proyección

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Comparar solo contra escaneo exagera el beneficio y oculta una solución más sencilla del motor.

Ejercicio

Añada primero un índice documental equivalente y mida antes de crear una copia.

Resultado que debe conservar

Exigir una línea base nativa competente antes de añadir una representación derivada.

Contrato mínimo

```
medir(consultaConÍndice) antesDe crearProyección()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida plan, filas, bytes, p95 y escritura. En la fase confirmatoria, C1 solo superó significativamente a B1 en cuatro de nueve celdas.

Índice antes que proyección

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar índice antes que proyección bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida plan, filas, bytes, p95 y escritura. En la fase confirmatoria, C1 solo superó significativamente a B1 en cuatro de nueve celdas.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Índice antes que proyección

Búsqueda de viveros por cantón

El equipo quiere copiar fichas completas cuando un índice compuesto basta.

Resultado: Se prueba primero una clave pequeña antes de mantener otra representación.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Búsqueda de viveros por cantón
type Solicitud = Record<string, unknown>;

class IndiceViveros {
  buscar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 19, evidencia: true, solicitud };
  }
}

const resultado = new IndiceViveros().buscar({ canton: "coto-
brus", especie: "roble-sabana" });
```

Python

```
# Búsqueda de viveros por cantón
class IndiceViveros:
    def buscar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 19, "evidencia": True, **solicitud}

resultado = IndiceViveros().buscar({"detalle": "canton: \"coto-
brus\", especie: \"roble-sabana\""})
```

Java

```
// Búsqueda de viveros por cantón
record Solicitud(String detalle) {}

final class IndiceViveros {
  Resultado buscar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(19, true, solicitud);
  }
}

var resultado = new IndiceViveros().buscar(new
Solicitud("canton: \"coto-brus\", especie: \"roble-sabana\""));
```

Go

```
// Búsqueda de viveros por cantón
type Solicitud struct { Detalle string }

func Buscar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 19, Evidencia: true}, nil
}

resultado, err := Buscar(Solicitud{Detalle: "canton: \"coto-
brus\", especie: \"roble-sabana\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

20 Retiro de proyección

Mecanismo de ciclo de vida | Evidencia: Contrato compuesto

La arquitectura conserva rutas útiles y retira las que ya no justifican su costo.

Propósito

Eliminar representaciones cuyo beneficio ya no compensa costo, riesgo o dependencia.

Problema

Las proyecciones se crean para consultas temporales y permanecen como escrituras, almacenamiento y código sin propietario.

Analogía

Es retirar una carretera con mantenimiento permanente cuando existe una ruta mejor y nadie la utiliza.

Solución y contrato

El inventario identifica consumidores, frecuencia, propietario, costo y sustituto. Se detienen escritores, se observa una ventana reversible y luego se retiran datos conforme a política.

Aplicación responsable

Actívelo ante cero uso, duplicación de índice, retraso persistente o ausencia de responsable. No borre antes de demostrar que los consumidores migraron.

Métricas, costos y relaciones

Mida consultas, costo evitado, dependientes y errores posteriores. Se relaciona con Registro de dependencias y Frontera lectura–escritura.

MECÁNICA · GOBERNANZA DEL CICLO DE VIDA

Cómo funciona Retiro de proyección

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
detenerConsumidores(); retirarProductor(); esperar(); eliminarDerivado()
```

> **Invariante principal.** Eliminar representaciones cuyo beneficio ya no compensa costo, riesgo o dependencia.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Eliminar representaciones cuyo beneficio ya no compensa costo, riesgo o dependencia.	Las proyecciones se crean para consultas temporales y permanecen como escrituras, almacenamiento y código sin propietario.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Identidad documental estable · Manifiesto de proyección · Repositorio con ámbito

LABORATORIO DE CAMPO · 20

Retiro de proyección

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Las proyecciones se crean para consultas temporales y permanecen como escrituras, almacenamiento y código sin propietario.

Ejercicio

Identifique una proyección sin uso, simule el retiro y verifique consumidores y reversión.

Resultado que debe conservar

Eliminar representaciones cuyo beneficio ya no compensa costo, riesgo o dependencia.

Contrato mínimo

```
detenerConsumidores(); retirarProductor(); esperar(); eliminarDerivado()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida consultas, costo evitado, dependientes y errores posteriores. Se relaciona con Registro de dependencias y Frontera lectura–escritura.

TARJETA DE REFERENCIA · 20

Retiro de proyección

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar retiro de proyección bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida consultas, costo evitado, dependientes y errores posteriores. Se relaciona con Registro de dependencias y Frontera lectura–escritura.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Retiro de proyección

Mapa antiguo que nadie consulta

Una proyección sin usuarios continúa consumiendo almacenamiento y monitoreo.

Resultado: Se confirma ausencia de consumidores, se conserva la fuente y se retira la copia.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Mapa antiguo que nadie consulta
type Solicitud = Record<string, unknown>;

class RetirarMapaHistorico {
  ejecutar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 20, evidencia: true, solicitud };
  }
}

const resultado = new RetirarMapaHistorico().ejecutar({
proyeccion: "mapa-2019", diasSinUso: 420 });
```

Python

```python
# Mapa antiguo que nadie consulta
class RetirarMapaHistorico:
    def ejecutar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 20, "evidencia": True, **solicitud}

resultado = RetirarMapaHistorico().ejecutar({"detalle":
"proyeccion: \"mapa-2019\", diasSinUso: 420"})
```

Java

```java
// Mapa antiguo que nadie consulta
record Solicitud(String detalle) {}

final class RetirarMapaHistorico {
  Resultado ejecutar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(20, true, solicitud);
  }
}

var resultado = new RetirarMapaHistorico().ejecutar(new
Solicitud("proyeccion: \"mapa-2019\", diasSinUso: 420"));
```

Go

```go
// Mapa antiguo que nadie consulta
type Solicitud struct { Detalle string }

func Ejecutar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 20, Evidencia: true}, nil
}

resultado, err := Ejecutar(Solicitud{Detalle: "proyeccion:
\"mapa-2019\", diasSinUso: 420"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

21 Identidad documental estable

Adaptación COBRIA | Evidencia: Identidad y migración

La identidad, la validación y las versiones convierten entradas dispersas en documentos interpretables.

Propósito

Mantener una identidad independiente de ubicación, forma y proveedor.

Problema

Claves derivadas de rutas, nombres visibles o posiciones cambian durante una migración y rompen referencias.

Analogía

Como el número de una muestra de campo: la etiqueta continúa aunque cambie de caja o estante.

Solución y contrato

Asigne una identidad opaca dentro del ámbito; trate claves externas como atributos versionados y valide colisiones en la frontera.

Aplicación responsable

Úselo cuando un documento sobreviva importaciones, fusiones o cambios de colección. Evite codificar significado mutable en la identidad.

Métricas, costos y relaciones

Mida colisiones, referencias huérfanas y tiempo de resolución. Se relaciona con Objeto canónico, Revisión obsoleta rechazada y Catálogo versionado.

MECÁNICA · MODELADO DOCUMENTAL

Cómo funciona Identidad documental estable

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
identidad = asignar(ámbito); referencias.usar(identidad)
```

> **Invariante principal.** Mantener una identidad independiente de ubicación, forma y proveedor.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Mantener una identidad independiente de ubicación, forma y proveedor.	Claves derivadas de rutas, nombres visibles o posiciones cambian durante una migración y rompen referencias.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Validación en frontera · Partición analítica sin fuga · Catálogo versionado

LABORATORIO DE CAMPO · 21

Identidad documental estable

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Claves derivadas de rutas, nombres visibles o posiciones cambian durante una migración y rompen referencias.

Ejercicio

Cambie ubicación y nombre externo de diez documentos y demuestre que sus referencias internas permanecen válidas.

Resultado que debe conservar

Mantener una identidad independiente de ubicación, forma y proveedor.

Contrato mínimo

```
identidad = asignar(ámbito); referencias.usar(identidad)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida colisiones, referencias huérfanas y tiempo de resolución. Se relaciona con Objeto canónico, Revisión obsoleta rechazada y Catálogo versionado.

TARJETA DE REFERENCIA · 21

Identidad documental estable

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar identidad documental estable bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida colisiones, referencias huérfanas y tiempo de resolución. Se relaciona con Objeto canónico, Revisión obsoleta rechazada y Catálogo versionado.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Identidad documental estable

La misma cámara cambia de ubicación

Usar la ubicación como identificador crea otro dispositivo al moverla.

Resultado: La identidad permanece estable mientras sus atributos pueden cambiar.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// La misma cámara cambia de ubicación
type Solicitud = Record<string, unknown>;

class CamaraTrampa {
  registrar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 21, evidencia: true, solicitud };
  }
}

const resultado = new CamaraTrampa().registrar({ camaraId: "cam-
0091", ubicacion: "sendero-norte" });
```

Python

```python
# La misma cámara cambia de ubicación
class CamaraTrampa:
    def registrar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 21, "evidencia": True, **solicitud}

resultado = CamaraTrampa().registrar({"detalle": "camaraId:
\"cam-0091\", ubicacion: \"sendero-norte\""})
```

Java

```java
// La misma cámara cambia de ubicación
record Solicitud(String detalle) {}

final class CamaraTrampa {
  Resultado registrar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(21, true, solicitud);
  }
}

var resultado = new CamaraTrampa().registrar(new
Solicitud("camaraId: \"cam-0091\", ubicacion: \"sendero-
norte\""));
```

Go

```go
// La misma cámara cambia de ubicación
type Solicitud struct { Detalle string }

func Registrar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 21, Evidencia: true}, nil
}

resultado, err := Registrar(Solicitud{Detalle: "camaraId: \"cam-
0091\", ubicacion: \"sendero-norte\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

22 Validación en frontera

Patrón establecido | Evidencia: Contrato verificable

La identidad, la validación y las versiones convierten entradas dispersas en documentos interpretables.

Propósito

Impedir que formas inválidas entren al núcleo documental.

Problema

Aceptar primero y corregir después dispersa supuestos y convierte cada lector en un validador distinto.

Analogía

Como una estación biológica: cada muestra se registra y revisa antes de entrar al archivo.

Solución y contrato

La frontera comprueba forma, ámbito, revisión, vocabulario y reglas invariantes; devuelve errores estructurados sin escritura parcial.

Aplicación responsable

Aplíquelo en API, importadores, consumidores y herramientas. Conserve el dato crudo separado cuando sea evidencia legítima.

Métricas, costos y relaciones

Mida rechazos por causa, campos desconocidos y escrituras parciales. Se conecta con Contrato versionado y Herramienta de agente gobernada.

MECÁNICA · MODELADO DOCUMENTAL

Cómo funciona Validación en frontera

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
validar(entrada, contrato, ámbito) o rechazarSinEfectos()
```

> **Invariante principal.** Impedir que formas inválidas entren al núcleo documental.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Impedir que formas inválidas entren al núcleo documental.	Aceptar primero y corregir después dispersa supuestos y convierte cada lector en un validador distinto.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Contrato documental versionado · Abstención con evidencia · Proyección de lectura

LABORATORIO DE CAMPO · 22

Validación en frontera

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Aceptar primero y corregir después dispersa supuestos y convierte cada lector en un validador distinto.

Ejercicio

Diseñe cinco entradas inválidas y compruebe rechazo completo, causa legible y ausencia de efectos parciales.

Resultado que debe conservar

Impedir que formas inválidas entren al núcleo documental.

Contrato mínimo

```
validar(entrada, contrato, ámbito) o rechazarSinEfectos()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida rechazos por causa, campos desconocidos y escrituras parciales. Se conecta con Contrato versionado y Herramienta de agente gobernada.

TARJETA DE REFERENCIA · 22

Validación en frontera

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar validación en frontera bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida rechazos por causa, campos desconocidos y escrituras parciales. Se conecta con Contrato versionado y Herramienta de agente gobernada.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Validación en frontera

Formulario ciudadano de avistamientos

Coordenadas imposibles o especies desconocidas llegan hasta el almacén.

Resultado: La entrada se normaliza o rechaza antes de crear el documento.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Formulario ciudadano de avistamientos
type Solicitud = Record<string, unknown>;

class IngresoAvistamiento {
  aceptar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 22, evidencia: true, solicitud };
  }
}

const resultado = new IngresoAvistamiento().aceptar({ latitud:
8.79, longitud: -82.96, especie: "tucan-pico-iris" });
```

Python

```python
# Formulario ciudadano de avistamientos
class IngresoAvistamiento:
    def aceptar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 22, "evidencia": True, **solicitud}

resultado = IngresoAvistamiento().aceptar({"detalle": "latitud:
8.79, longitud: -82.96, especie: \"tucan-pico-iris\""})
```

Java

```java
// Formulario ciudadano de avistamientos
record Solicitud(String detalle) {}

final class IngresoAvistamiento {
  Resultado aceptar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(22, true, solicitud);
  }
}

var resultado = new IngresoAvistamiento().aceptar(new
Solicitud("latitud: 8.79, longitud: -82.96, especie: \"tucan-
pico-iris\""));
```

Go

```go
// Formulario ciudadano de avistamientos
type Solicitud struct { Detalle string }

func Aceptar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 22, Evidencia: true}, nil
}

resultado, err := Aceptar(Solicitud{Detalle: "latitud: 8.79,
longitud: -82.96, especie: \"tucan-pico-iris\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

23 Contrato documental versionado

Adaptación COBRIA | Evidencia: Compatibilidad demostrada

La identidad, la validación y las versiones convierten entradas dispersas en documentos interpretables.

Propósito

Evolucionar documentos sin exigir una migración instantánea de todo el conjunto.

Problema

Cambiar el significado de un campo bajo el mismo nombre produce lectores temporalmente incompatibles.

Analogía

Como una guía de especies por ediciones: cada observación indica con qué edición fue clasificada.

Solución y contrato

Cada documento declara versión; lectores admiten un intervalo explícito y las transformaciones entre versiones son deterministas e idempotentes.

Aplicación responsable

Úselo ante cambios aditivos o incompatibles. Mantenga pruebas de lectura hacia atrás y hacia delante durante la convivencia.

Métricas, costos y relaciones

Mida documentos por versión, fallos de conversión y consumidores antiguos. Se relaciona con Catálogo versionado y Publicación por versión.

MECÁNICA · MODELADO DOCUMENTAL

Cómo funciona Contrato documental versionado

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
leer(según versión); migrar(de, hacia) deFormaIdempotente()
```

> **Invariante principal.** Evolucionar documentos sin exigir una migración instantánea de todo el conjunto.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Evolucionar documentos sin exigir una migración instantánea de todo el conjunto.	Cambiar el significado de un campo bajo el mismo nombre produce lectores temporalmente incompatibles.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Lectura coherente por revisión · Presupuesto de derivados · Proyección de cobertura

LABORATORIO DE CAMPO · 23

Contrato documental versionado

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Cambiar el significado de un campo bajo el mismo nombre produce lectores temporalmente incompatibles.

Ejercicio

Lea documentos de tres versiones, migre dos veces la misma muestra y compare huellas.

Resultado que debe conservar

Evolucionar documentos sin exigir una migración instantánea de todo el conjunto.

Contrato mínimo

```
leer(según versión); migrar(de, hacia) deFormaIdempotente()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida documentos por versión, fallos de conversión y consumidores antiguos. Se relaciona con Catálogo versionado y Publicación por versión.

Contrato documental versionado

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar contrato documental versionado bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida documentos por versión, fallos de conversión y consumidores antiguos. Se relaciona con Catálogo versionado y Publicación por versión.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Contrato documental versionado

Aplicación móvil todavía usa el formato anterior

Cambiar nombre de campos rompe tabletas que aún no se actualizaron.

Resultado: El lector selecciona el contrato correcto y migra de forma explícita.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Aplicación móvil todavía usa el formato anterior
type Solicitud = Record<string, unknown>;

class ContratoObservacion {
  decodificar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 23, evidencia: true, solicitud };
  }
}

const resultado = new ContratoObservacion().decodificar({
schemaVersion: 2, especieCodigo: "AV-12" });
```

Python

```python
# Aplicación móvil todavía usa el formato anterior
class ContratoObservacion:
    def decodificar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 23, "evidencia": True, **solicitud}

resultado = ContratoObservacion().decodificar({"detalle":
"schemaVersion: 2, especieCodigo: \"AV-12\""})
```

Java

```java
// Aplicación móvil todavía usa el formato anterior
record Solicitud(String detalle) {}

final class ContratoObservacion {
  Resultado decodificar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(23, true, solicitud);
  }
}

var resultado = new ContratoObservacion().decodificar(new
Solicitud("schemaVersion: 2, especieCodigo: \"AV-12\""));
```

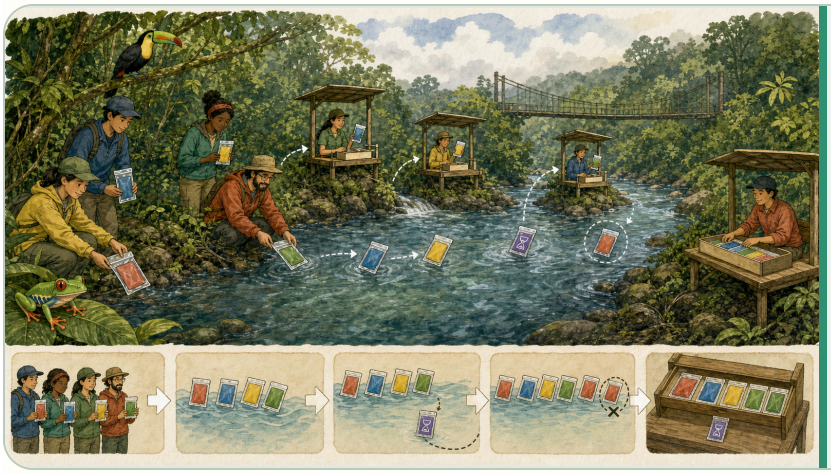
Go

```go
// Aplicación móvil todavía usa el formato anterior
type Solicitud struct { Detalle string }

func Decodificar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 23, Evidencia: true}, nil
}

resultado, err := Decodificar(Solicitud{Detalle: "schemaVersion:
2, especieCodigo: \"AV-12\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

24 Lectura coherente por revisión

Contrato compuesto | Evidencia: Consistencia observable



Eventos repetidos, tardíos y ordenados atraviesan una ruta observable.

Propósito

Evitar respuestas formadas por fragmentos de revisiones incompatibles.

Problema

Consultar varias proyecciones puede mezclar estados aunque cada lectura aislada sea correcta.

Analogía

Como comparar fotografías de una misma parcela: todas deben pertenecer al mismo levantamiento.

Solución y contrato

Propague revisión objetivo o corte; acepte solo fragmentos compatibles y declare degradación cuando no pueda completarse.

Aplicación responsable

Aplíquelo en vistas compuestas, informes y recuperación aumentada. No prometa coherencia que el motor o las proyecciones no sostienen.

Métricas, costos y relaciones

Mida mezclas detectadas, abstenciones y atraso por revisión. Se vincula con Reconstrucción incremental y Recuperación limitada por ámbito.

MECÁNICA · EVENTOS Y CONSISTENCIA

Cómo funciona Lectura coherente por revisión

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
leerTodoEn(revisiónObjetivo) o declararDegradación()
```

> **Invariante principal.** Evitar respuestas formadas por fragmentos de revisiones incompatibles.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Evitar respuestas formadas por fragmentos de revisiones incompatibles.	Consultar varias proyecciones puede mezclar estados aunque cada lectura aislada sea correcta.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Ventana de frescura declarada · Auditoría narrativa reconstruible · Reconstrucción total

LABORATORIO DE CAMPO · 24

Lectura coherente por revisión

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Consultar varias proyecciones puede mezclar estados aunque cada lectura aislada sea correcta.

Ejercicio

Combine dos proyecciones con atraso distinto y compruebe que nunca se presenta una revisión mezclada.

Resultado que debe conservar

Evitar respuestas formadas por fragmentos de revisiones incompatibles.

Contrato mínimo

```
leerTodoEn(revisiónObjetivo) o declararDegradación()
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida mezclas detectadas, abstenciones y atraso por revisión. Se vincula con Reconstrucción incremental y Recuperación limitada por ámbito.

TARJETA DE REFERENCIA · 24

Lectura coherente por revisión

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar lectura coherente por revisión bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida mezclas detectadas, abstenciones y atraso por revisión. Se vincula con Reconstrucción incremental y Recuperación limitada por ámbito.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Lectura coherente por revisión

Informe combina mapa y estadísticas

Leer partes en momentos distintos produce un informe internamente contradictorio.

Resultado: Todas las piezas declaran la misma revisión o la lectura se repite.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Informe combina mapa y estadísticas
type Solicitud = Record<string, unknown>;

class InformeReserva {
  cargar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 24, evidencia: true, solicitud };
  }
}

const resultado = new InformeReserva().cargar({ reservaId:
"corcovado", revision: 884 });
```

Python

```
# Informe combina mapa y estadísticas
class InformeReserva:
    def cargar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 24, "evidencia": True, **solicitud}

resultado = InformeReserva().cargar({"detalle": "reservaId:
\"corcovado\", revision: 884"})
```

Java

```
// Informe combina mapa y estadísticas
record Solicitud(String detalle) {}

final class InformeReserva {
  Resultado cargar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(24, true, solicitud);
  }
}

var resultado = new InformeReserva().cargar(new
Solicitud("reservaId: \"corcovado\", revision: 884"));
```

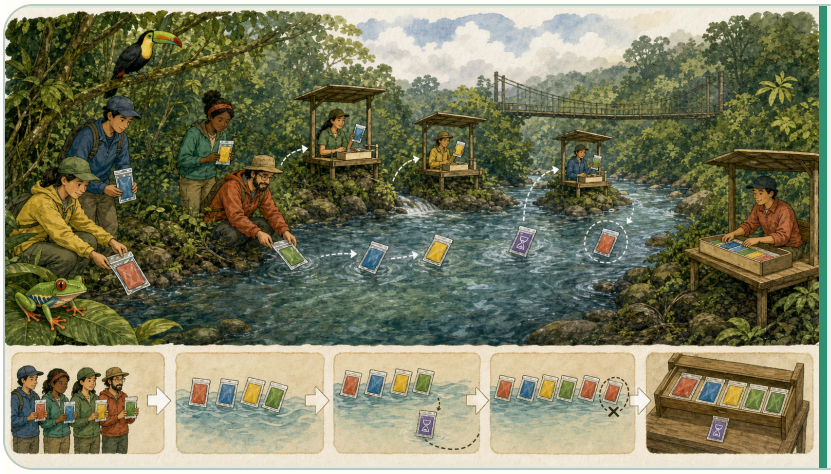
Go

```
// Informe combina mapa y estadísticas
type Solicitud struct { Detalle string }

func Cargar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 24, Evidencia: true}, nil
}

resultado, err := Cargar(Solicitud{Detalle: "reservaId:
\"corcovado\", revision: 884"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

25 Ventana de frescura declarada

Adaptación COBRIA | Evidencia: Acuerdo medible



Eventos repetidos, tardíos y ordenados atraviesan una ruta observable.

Propósito

Hacer explícito cuánto atraso puede tolerar cada lectura.

Problema

Sin un límite declarado, una proyección aparentemente rápida puede entregar información inaceptablemente antigua.

Analogía

Como un boletín meteorológico: su utilidad depende de cuándo fue observado y cuándo se consulta.

Solución y contrato

Cada proyección publica última revisión aplicada, tiempo de actualización y límite contratado; el lector decide esperar, degradar o fallar.

Aplicación responsable

Úselo en derivados asíncronos. Defina ventanas por decisión, no una cifra universal para todo el sistema.

Métricas, costos y relaciones

Mida edad, percentiles de atraso e incumplimientos. Se conecta con Reconstrucción incremental y Frontera lectura-escritura.

MECÁNICA · EVENTOS Y CONSISTENCIA

Cómo funciona Ventana de frescura declarada

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
si edad > ventana entonces esperar | degradar | fallar
```

> **Invariante principal.** Hacer explícito cuánto atraso puede tolerar cada lectura.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Hacer explícito cuánto atraso puede tolerar cada lectura.	Sin un límite declarado, una proyección aparentemente rápida puede entregar información inaceptablemente antigua.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Manifiesto de proyección · Objeto canónico · Reconstrucción incremental

LABORATORIO DE CAMPO · 25

Ventana de frescura declarada

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Sin un límite declarado, una proyección aparentemente rápida puede entregar información inaceptablemente antigua.

Ejercicio

Supere deliberadamente la ventana de frescura y verifique espera, degradación y fallo según contrato.

Resultado que debe conservar

Hacer explícito cuánto atraso puede tolerar cada lectura.

Contrato mínimo

```
si edad > ventana entonces esperar | degradar | fallar
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida edad, percentiles de atraso e incumplimientos. Se conecta con Reconstrucción incremental y Frontera lectura-escritura.

Ventana de frescura declarada

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar ventana de frescura declarada bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida edad, percentiles de atraso e incumplimientos. Se conecta con Reconstrucción incremental y Frontera lectura-escritura.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Ventana de frescura declarada

Tablero de incendios forestales

Un tablero parece actual aunque la última proyección tenga dos horas.

Resultado: El consumidor acepta, espera o rechaza según la edad declarada.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Tablero de incendios forestales
type Solicitud = Record<string, unknown>;

class TableroIncendios {
  consultar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 25, evidencia: true, solicitud };
  }
}

const resultado = new TableroIncendios().consultar({ region:
"brunca", frescuraMaxMin: 10 });
```

Python

```python
# Tablero de incendios forestales
class TableroIncendios:
    def consultar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 25, "evidencia": True, **solicitud}

resultado = TableroIncendios().consultar({"detalle": "region:
\"brunca\", frescuraMaxMin: 10"})
```

Java

```java
// Tablero de incendios forestales
record Solicitud(String detalle) {}

final class TableroIncendios {
  Resultado consultar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(25, true, solicitud);
  }
}

var resultado = new TableroIncendios().consultar(new
Solicitud("region: \"brunca\", frescuraMaxMin: 10"));
```

Go

```go
// Tablero de incendios forestales
type Solicitud struct { Detalle string }

func Consultar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 25, Evidencia: true}, nil
}

resultado, err := Consultar(Solicitud{Detalle: "region:
\"brunca\", frescuraMaxMin: 10"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

26 Manifiesto de proyección

Adaptación COBRIA | Evidencia: Procedencia completa

Una representación derivada vuelve a producirse y se compara con la fuente.

Propósito

Explicar exactamente con qué fuentes y transformación se produjo un derivado.

Problema

Una copia puede parecer correcta y aun mezclar semillas, versiones o cortes distintos.

Analogía

Como la ficha de una colección científica: origen, fecha, método y responsable acompañan al conjunto.

Solución y contrato

Guarde fuentes, cortes, versiones, parámetros, semilla, huellas y resultado de verificación como manifiesto inmutable.

Aplicación responsable

Úselo en índices, conjuntos analíticos y publicaciones. El manifiesto referencia secretos sin copiarlos.

Métricas, costos y relaciones

Mida manifiestos incompletos, huellas inválidas y tiempo de reproducción. Se relaciona con Conjunto reproducible y Reparación verificable.

MECÁNICA · RECONSTRUCCIÓN

Cómo funciona Manifiesto de proyección

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
manifiesto = sellar(fuentes, cortes, versiones, semilla, huellas)
```

> **Invariante principal.** Explicar exactamente con qué fuentes y transformación se produjo un derivado.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Explicar exactamente con qué fuentes y transformación se produjo un derivado.	Una copia puede parecer correcta y aun mezclar semillas, versiones o cortes distintos.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Partición analítica sin fuga · Dato fechado · Publicación por versión

LABORATORIO DE CAMPO · 26

Manifiesto de proyección

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Una copia puede parecer correcta y aun mezclar semillas, versiones o cortes distintos.

Ejercicio

Reconstruya una proyección únicamente desde su manifiesto y registre cualquier dato implícito faltante.

Resultado que debe conservar

Explicar exactamente con qué fuentes y transformación se produjo un derivado.

Contrato mínimo

```
manifiesto = sellar(fuentes, cortes, versiones, semilla, huellas)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida manifiestos incompletos, huellas inválidas y tiempo de reproducción. Se relaciona con Conjunto reproducible y Reparación verificable.

TARJETA DE REFERENCIA · 26

Manifiesto de proyección

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar manifiesto de proyección bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida manifiestos incompletos, huellas inválidas y tiempo de reproducción. Se relaciona con Conjunto reproducible y Reparación verificable.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Manifiesto de proyección

Mapa de corredores biológicos

Nadie recuerda con qué fuentes y filtros se produjo el mapa publicado.

Resultado: La proyección publica receta, entradas, versión, huellas y responsable.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Mapa de corredores biológicos
type Solicitud = Record<string, unknown>;

class ManifiestoCorredores {
  crear(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 26, evidencia: true, solicitud };
  }
}

const resultado = new ManifiestoCorredores().crear({ version:
"corredores-12", fuentes: ["bosque-v7", "rios-v3"] });
```

Python

```
# Mapa de corredores biológicos
class ManifiestoCorredores:
    def crear(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 26, "evidencia": True, **solicitud}

resultado = ManifiestoCorredores().crear({"detalle": "version:
\"corredores-12\", fuentes: [\"bosque-v7\", \"rios-v3\"]"})
```

Java

```
// Mapa de corredores biológicos
record Solicitud(String detalle) {}

final class ManifiestoCorredores {
  Resultado crear(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(26, true, solicitud);
  }
}

var resultado = new ManifiestoCorredores().crear(new
Solicitud("version: \"corredores-12\", fuentes: [\"bosque-v7\",
\"rios-v3\"]"));
```

Go

```
// Mapa de corredores biológicos
type Solicitud struct { Detalle string }

func Crear(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 26, Evidencia: true}, nil
}

resultado, err := Crear(Solicitud{Detalle: "version:
\"corredores-12\", fuentes: [\"bosque-v7\", \"rios-v3\"]"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

27 Partición analítica sin fuga

Contrato compuesto | Evidencia: Evaluación válida

Particiones, transformaciones y resultados mantienen un linaje visible.

Propósito

Separar entrenamiento, validación y prueba sin contaminar tiempo, entidad ni ámbito.

Problema

Una partición aleatoria puede colocar revisiones de la misma entidad a ambos lados y exagerar la calidad del modelo.

Analogía

Como estudiar aves por rutas distintas: no se cuenta la misma bandada como descubrimiento independiente.

Solución y contrato

Particione primero por entidad y tiempo, luego por ámbito; congele reglas y audite intersecciones antes de calcular métricas.

Aplicación responsable

Úselo en minería de datos y aprendizaje automático. Documente exclusiones y conserve una prueba negativa de contaminación.

Métricas, costos y relaciones

Mida intersecciones, deriva temporal y rendimiento por ámbito. Se conecta con Conjunto reproducible y Característica versionada.

MECÁNICA · ANALÍTICA REPRODUCIBLE

Cómo funciona Partición analítica sin fuga

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
partirPor(entidad, tiempo, ámbito); exigir(intersección = ∅)
```

> **Invariante principal.** Separar entrenamiento, validación y prueba sin contaminar tiempo, entidad ni ámbito.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Separar entrenamiento, validación y prueba sin contaminar tiempo, entidad ni ámbito.	Una partición aleatoria puede colocar revisiones de la misma entidad a ambos lados y exagerar la calidad del modelo.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Abstención con evidencia · Repositorio con ámbito · Procesamiento idempotente

LABORATORIO DE CAMPO · 27

Partición analítica sin fuga

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Una partición aleatoria puede colocar revisiones de la misma entidad a ambos lados y exagerar la calidad del modelo.

Ejercicio

Particione un conjunto por entidad y tiempo; escriba una prueba que falle ante cualquier intersección.

Resultado que debe conservar

Separar entrenamiento, validación y prueba sin contaminar tiempo, entidad ni ámbito.

Contrato mínimo

```
partirPor(entidad, tiempo, ámbito); exigir(intersección = ∅)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida intersecciones, deriva temporal y rendimiento por ámbito. Se conecta con Conjunto reproducible y Característica versionada.

Partición analítica sin fuga

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar partición analítica sin fuga bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida intersecciones, deriva temporal y rendimiento por ámbito. Se conecta con Conjunto reproducible y Característica versionada.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Partición analítica sin fuga

Predicción de floración

Observaciones futuras del mismo árbol aparecen en entrenamiento y validación.

Resultado: La partición respeta grupos y tiempo antes de calcular características.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```
// Predicción de floración
type Solicitud = Record<string, unknown>;

class ParticionFloracion {
  dividir(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 27, evidencia: true, solicitud };
  }
}

const resultado = new ParticionFloracion().dividir({ grupo:
"arbolId", corte: "2025-12-31", semilla: 73 });
```

Python

```
# Predicción de floración
class ParticionFloracion:
    def dividir(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 27, "evidencia": True, **solicitud}

resultado = ParticionFloracion().dividir({"detalle": "grupo:
\"arbolId\", corte: \"2025-12-31\", semilla: 73"})
```

Java

```
// Predicción de floración
record Solicitud(String detalle) {}

final class ParticionFloracion {
  Resultado dividir(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(27, true, solicitud);
  }
}

var resultado = new ParticionFloracion().dividir(new
Solicitud("grupo: \"arbolId\", corte: \"2025-12-31\", semilla:
73"));
```

Go

```
// Predicción de floración
type Solicitud struct { Detalle string }

func Dividir(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 27, Evidencia: true}, nil
}

resultado, err := Dividir(Solicitud{Detalle: "grupo:
\"arbolId\", corte: \"2025-12-31\", semilla: 73"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

28 Abstención con evidencia

Control de IA | Evidencia: Seguridad y honestidad

La respuesta enlaza evidencia autorizada o se abstiene cuando no la posee.

Propósito

Responder solo cuando la evidencia autorizada sostiene la salida.

Problema

Un sistema generativo puede completar vacíos con lenguaje plausible o citar fragmentos inexistentes.

Analogía

Como una guía responsable: si no observa señales suficientes, declara que no puede identificar la especie.

Solución y contrato

Defina umbrales, cobertura mínima, citas válidas y causas de abstención; la generación recibe únicamente evidencia autorizada.

Aplicación responsable

Úselo en asistentes y automatizaciones. La abstención debe ser útil: explique qué falta sin revelar material prohibido.

Métricas, costos y relaciones

Mida precisión, cobertura, citas válidas, fugas y abstención apropiada. Se relaciona con Recuperación limitada por ámbito.

MECÁNICA · IA CON PROCEDENCIA

Cómo funciona Abstención con evidencia

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
si evidenciaInsuficiente entonces abstener(causaSegura)
```

> **Invariante principal.** Responder solo cuando la evidencia autorizada sostiene la salida.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Responder solo cuando la evidencia autorizada sostiene la salida.	Un sistema generativo puede completar vacíos con lenguaje plausible o citar fragmentos inexistentes.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Presupuesto de derivados · Catálogo versionado · Revisión obsoleta rechazada

LABORATORIO DE CAMPO · 28

Abstención con evidencia

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Un sistema generativo puede completar vacíos con lenguaje plausible o citar fragmentos inexistentes.

Ejercicio

Retire evidencia suficiente y compruebe que la respuesta se abstiene con una explicación segura.

Resultado que debe conservar

Responder solo cuando la evidencia autorizada sostiene la salida.

Contrato mínimo

```
si evidenciaInsuficiente entonces abstener(causaSegura)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida precisión, cobertura, citas válidas, fugas y abstención apropiada. Se relaciona con Recuperación limitada por ámbito.

TARJETA DE REFERENCIA · 28

Abstención con evidencia

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar abstención con evidencia bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida precisión, cobertura, citas válidas, fugas y abstención apropiada. Se relaciona con Recuperación limitada por ámbito.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Abstención con evidencia

Asistente identifica una especie en una foto borrosa

El modelo podría inventar una respuesta segura sin evidencia suficiente.

Resultado: El asistente se abstiene, explica qué falta y conserva candidatos.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Asistente identifica una especie en una foto borrosa
type Solicitud = Record<string, unknown>;

class IdentificadorEspecies {
  responder(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 28, evidencia: true, solicitud };
  }
}

const resultado = new IdentificadorEspecies().responder({
confianza: 0.41, minimo: 0.78, candidatos: ["tangara",
"mielero"] });
```

Python

```python
# Asistente identifica una especie en una foto borrosa
class IdentificadorEspecies:
    def responder(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 28, "evidencia": True, **solicitud}

resultado = IdentificadorEspecies().responder({"detalle":
"confianza: 0.41, minimo: 0.78, candidatos: [\"tangara\",
\"mielero\"]"})
```

Java

```java
// Asistente identifica una especie en una foto borrosa
record Solicitud(String detalle) {}

final class IdentificadorEspecies {
  Resultado responder(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(28, true, solicitud);
  }
}

var resultado = new IdentificadorEspecies().responder(new
Solicitud("confianza: 0.41, minimo: 0.78, candidatos:
[\"tangara\", \"mielero\"]"));
```

Go

```go
// Asistente identifica una especie en una foto borrosa
type Solicitud struct { Detalle string }

func Responder(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 28, Evidencia: true}, nil
}

resultado, err := Responder(Solicitud{Detalle: "confianza: 0.41,
minimo: 0.78, candidatos: [\"tangara\", \"mielero\"]"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

29 Presupuesto de derivados

Adaptación COBRIA | Evidencia: Costo gobernado

La arquitectura conserva rutas útiles y retira las que ya no justifican su costo.

Propósito

Limitar cuántas copias optimizadas puede sostener responsablemente un sistema.

Problema

Cada proyección mejora una lectura local, pero añade escritura, almacenamiento, monitoreo y carga cognitiva.

Analogía

Como senderos en una reserva: cada nuevo recorrido necesita propósito, mantenimiento y cierre seguro.

Solución y contrato

Toda propuesta declara beneficio, costo total, propietario, fecha de revisión y criterio de retiro antes de ser publicada.

Aplicación responsable

Úselo al diseñar o revisar proyecciones. Agrupe necesidades compatibles antes de crear otra copia.

Métricas, costos y relaciones

Mida uso, costo por consulta, atraso e incidentes. Se conecta con Frontera lectura-escritura y Retiro de proyección.

MECÁNICA · GOBERNANZA DEL CICLO DE VIDA

Cómo funciona Presupuesto de derivados

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
autorizarDerivado soloSi beneficio > costoTotal y tieneRetiro
```

> **Invariante principal.** Limitar cuántas copias optimizadas puede sostener responsablemente un sistema.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Limitar cuántas copias optimizadas puede sostener responsablemente un sistema.	Cada proyección mejora una lectura local, pero añade escritura, almacenamiento, monitoreo y carga cognitiva.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Auditoría narrativa reconstruible · Proyección de lectura · Cola de salida transaccional

Presupuesto de derivados

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Cada proyección mejora una lectura local, pero añade escritura, almacenamiento, monitoreo y carga cognitiva.

Ejercicio

Calcule el costo anual de tres derivados y retire o combine el que no justifique su mantenimiento.

Resultado que debe conservar

Limitar cuántas copias optimizadas puede sostener responsablemente un sistema.

Contrato mínimo

```
autorizarDerivado soloSi beneficio > costoTotal y tieneRetiro
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida uso, costo por consulta, atraso e incidentes. Se conecta con Frontera lectura-escritura y Retiro de proyección.

Presupuesto de derivados

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar presupuesto de derivados bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida uso, costo por consulta, atraso e incidentes. Se conecta con Frontera lectura-escritura y Retiro de proyección.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Presupuesto de derivados

Cada equipo solicita su propio tablero

Diez proyecciones similares multiplican escritura, costo y mantenimiento.

Resultado: La copia solo se aprueba con propietario, beneficio y criterio de retiro.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Cada equipo solicita su propio tablero
type Solicitud = Record<string, unknown>;

class PresupuestoProyecciones {
  evaluar(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 29, evidencia: true, solicitud };
  }
}

const resultado = new PresupuestoProyecciones().evaluar({
beneficioMensual: 18, costoMensual: 31, consumidores: 1 });
```

Python

```python
# Cada equipo solicita su propio tablero
class PresupuestoProyecciones:
    def evaluar(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 29, "evidencia": True, **solicitud}

resultado = PresupuestoProyecciones().evaluar({"detalle":
"beneficioMensual: 18, costoMensual: 31, consumidores: 1"})
```

Java

```java
// Cada equipo solicita su propio tablero
record Solicitud(String detalle) {}

final class PresupuestoProyecciones {
  Resultado evaluar(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(29, true, solicitud);
  }
}

var resultado = new PresupuestoProyecciones().evaluar(new
Solicitud("beneficioMensual: 18, costoMensual: 31, consumidores:
1"));
```

Go

```go
// Cada equipo solicita su propio tablero
type Solicitud struct { Detalle string }

func Evaluar(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 29, Evidencia: true}, nil
}

resultado, err := Evaluar(Solicitud{Detalle: "beneficioMensual:
18, costoMensual: 31, consumidores: 1"})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

30 Auditoría narrativa reconstruible

Experimental | Evidencia: Explicación reproducible

La identidad, la validación y las versiones convierten entradas dispersas en documentos interpretables.

Propósito

Convertir eventos técnicos en una historia verificable de decisión y cambio.

Problema

Los registros aislados muestran acciones, pero no siempre explican quién autorizó, qué versión se usó y qué consecuencia ocurrió.

Analogía

Como un cuaderno de campo: cada observación enlaza lugar, momento, responsable y evidencia.

Solución y contrato

Componga una vista temporal desde eventos inmutables; mantenga enlaces al documento, revisión, ámbito, política y resultado, sin volverla autoridad.

Aplicación responsable

Úselo en incidentes, decisiones asistidas y revisión regulatoria. Proteja datos sensibles y aplique retención diferenciada.

Métricas, costos y relaciones

Mida eventos sin enlace, secuencias imposibles y tiempo de explicación. Se conecta con Dato fechado, Cola de salida y Herramienta gobernada.

MECÁNICA · MODELADO DOCUMENTAL

Cómo funciona Auditoría narrativa reconstruible

La estructura convierte el propósito en responsabilidades separadas y comprobables.

Secuencia

1. El consumidor expresa identidad, ámbito, revisión y necesidad de lectura.
2. El contrato valida precondiciones antes de tocar la colección.
3. El repositorio recupera o modifica únicamente el documento autorizado.
4. El mecanismo produce evidencia versionada y nunca suplanta la autoridad.
5. El verificador compara resultado, límites y caso negativo.

Pseudocódigo neutral

```
narrar(eventosOrdenados); exigirEnlace(documento, revisión, política)
```

> **Invariante principal.** Convertir eventos técnicos en una historia verificable de decisión y cambio.

DECIDIR CON CRITERIO

Cuándo usarlo y cuándo evitarlo

Conviene cuando

- la tensión descrita aparece en mediciones o incidentes;
- existe una persona responsable del contrato;
- puede probarse reconstrucción, rechazo o abstención;
- el beneficio supera el costo operativo completo.

No conviene cuando

- un índice o validación más simple resuelve la necesidad;
- la copia no tiene consumidor ni límite de frescura;
- se intenta ocultar una autoridad ambigua;
- no existe mecanismo de reversión o retiro.

Consecuencias

Beneficio	Costo o riesgo	Control
Convertir eventos técnicos en una historia verificable de decisión y cambio.	Los registros aislados muestran acciones, pero no siempre explican quién autorizó, qué versión se usó y qué consecuencia ocurrió.	Contrato, métrica y prueba negativa
Responsabilidad visible	Más disciplina documental	Rúbrica y revisión periódica

Errores frecuentes

Aplicarlo por nombre, omitir el ámbito, aceptar revisiones implícitas o medir únicamente el caso feliz. La solución se considera incompleta si no puede explicar su fallo.

Conecta con: Objeto canónico · Proyección de cobertura · Reparación verificable

LABORATORIO DE CAMPO · 30

Auditoría narrativa reconstruible

Ahora convierta la idea en una prueba pequeña, observable y reversible.

Situación para diagnosticar

Los registros aislados muestran acciones, pero no siempre explican quién autorizó, qué versión se usó y qué consecuencia ocurrió.

Ejercicio

Reconstruya la narración de un incidente desde eventos y señale cada afirmación sin evidencia enlazada.

Resultado que debe conservar

Convertir eventos técnicos en una historia verificable de decisión y cambio.

Contrato mínimo

```
narrar(eventosOrdenados); exigirEnlace(documento, revisión, política)
```

Preguntas de revisión

- ¿Quién posee la autoridad?
- ¿Dónde aparece el ámbito y la versión?
- ¿Cómo se prueba el fallo?
- ¿Cómo se revierte o reconstruye?

Evidencia para guardar

- manifiesto y versión;
- entrada y resultado esperado;
- métrica y límite;
- captura del caso negativo.

Conexiones

Mida eventos sin enlace, secuencias imposibles y tiempo de explicación. Se conecta con Dato fechado, Cola de salida y Herramienta gobernada.

TARJETA DE REFERENCIA · 30

Auditoría narrativa reconstruible

Contrato de aceptación

1. Preparar un documento válido y otro inválido.
2. Ejecutar auditoría narrativa reconstruible bajo un ámbito conocido.
3. Comprobar resultado, revisión y procedencia.
4. Repetir, interrumpir o reconstruir según corresponda.
5. Guardar manifiesto, métricas y causa de cualquier exclusión.

Lista de comprobación

- ☐ Autoridad identificada
- ☐ Ámbito obligatorio
- ☐ Contrato versionado
- ☐ Caso negativo probado
- ☐ Observabilidad suficiente
- ☐ Reversión o reconstrucción

Señales que debe observar

Mida eventos sin enlace, secuencias imposibles y tiempo de explicación. Se conecta con Dato fechado, Cola de salida y Herramienta gobernada.

Preguntas para revisión de diseño

¿Qué afirmación concreta vuelve verdadera este patrón?

¿Qué dato demuestra que continúa funcionando?

¿Qué ocurre si se repite, atrasa o interrumpe?

¿Cómo se retira sin perder la fuente?

Regla de salida. Si no puede responder las cuatro preguntas con evidencia, el patrón aún no está implementado.

Auditoría narrativa reconstruible

Explicar por qué se cerró una reserva

Registros aislados no cuentan qué lluvia, regla y aprobación produjeron la decisión.

Resultado: Una línea temporal enlaza eventos, revisiones, política, responsable y resultado.

El mismo caso se expresa en cuatro lenguajes. Identifique participantes y contrato antes de adaptar el ejemplo a su almacén documental.

TypeScript

```typescript
// Explicar por qué se cerró una reserva
type Solicitud = Record<string, unknown>;

class HistoriaCierreReserva {
  reconstruir(solicitud: Solicitud) {
    if (!solicitud) throw new Error("Solicitud requerida");
    return { patron: 30, evidencia: true, solicitud };
  }
}

const resultado = new HistoriaCierreReserva().reconstruir({
reservaId: "la-amistad", desde: "2026-08-23T18:00:00Z" });
```

Python

```python
# Explicar por qué se cerró una reserva
class HistoriaCierreReserva:
    def reconstruir(self, solicitud):
        if not solicitud:
            raise ValueError("Solicitud requerida")
        return {"patron": 30, "evidencia": True, **solicitud}

resultado = HistoriaCierreReserva().reconstruir({"detalle":
"reservaId: \"la-amistad\", desde: \"2026-08-23T18:00:00Z\""})
```

Java

```java
// Explicar por qué se cerró una reserva
record Solicitud(String detalle) {}

final class HistoriaCierreReserva {
  Resultado reconstruir(Solicitud solicitud) {
    if (solicitud == null) throw new IllegalArgumentException();
    return new Resultado(30, true, solicitud);
  }
}

var resultado = new HistoriaCierreReserva().reconstruir(new
Solicitud("reservaId: \"la-amistad\", desde: \"2026-08-
23T18:00:00Z\""));
```

Go

```go
// Explicar por qué se cerró una reserva
type Solicitud struct { Detalle string }

func Reconstruir(s Solicitud) (Resultado, error) {
  if s.Detalle == "" { return Resultado{}, errors.New("solicitud
requerida") }
  return Resultado{Patron: 30, Evidencia: true}, nil
}

resultado, err := Reconstruir(Solicitud{Detalle: "reservaId:
\"la-amistad\", desde: \"2026-08-23T18:00:00Z\""})
```

Prueba mínima. Ejecute el caso válido, elimine el dato que delimita la operación y compruebe que el fallo conserva evidencia.

CASO GUIADO 1

Reservas y servicios

Una organización administra alojamientos, disponibilidad, servicios y cobros documentales.

> **Patrones principales.** Objeto canónico, Catálogo versionado, Proyección de lectura, Cola de salida transaccional, Ventana de frescura declarada.

CASO 1 · DISEÑO

Del lenguaje del dominio al documento

Decisiones

1. Definir identidad y ámbito antes de elegir colecciones.
2. Separar hechos históricos de estados reemplazables.
3. Versionar vocabularios y contratos desde la primera migración.
4. Diseñar cada proyección desde una consulta concreta.

Documento mínimo

```
{ identidad, ámbito, revisión, contrato, datos, procedencia }
```

> La forma final cambia con el dominio; los campos de gobierno permanecen verificables.

CASO 1 · FALLOS

Ensayo de degradación

Fallo	Respuesta esperada
Evento duplicado	Un solo efecto por revisión
Proyección atrasada	Frescura visible y degradación declarada
Ámbito manipulado	Rechazo antes de recuperar documentos
Cambio incompatible	Nueva versión y convivencia controlada

Evidencia

Guarde manifiesto, entradas, revisión inicial y final, métricas, caso negativo y procedimiento de reversión.

CASO 1 · REVISIÓN

Preguntas para el equipo

¿Dónde vive la única autoridad de cambio?

¿Qué copia puede borrarse hoy?

¿Qué lectura tolera atraso y cuánto?

¿Qué dato no debe cruzar un ámbito?

¿Cómo se reproduce una cifra analítica?

¿Qué mecanismo podría retirarse?

Cierre. El caso termina cuando otra persona puede reconstruir la decisión sin preguntarle al autor.

CASO GUIADO 2

Seguimiento deportivo

Un programa registra sesiones, planteles, métricas fechadas y análisis longitudinal.

> **Patrones principales.** Dato fechado, Repositorio con ámbito, Conjunto reproducible, Característica versionada, Partición sin fuga.

CASO 2 · DISEÑO

Del lenguaje del dominio al documento

Decisiones

1. Definir identidad y ámbito antes de elegir colecciones.
2. Separar hechos históricos de estados reemplazables.
3. Versionar vocabularios y contratos desde la primera migración.
4. Diseñar cada proyección desde una consulta concreta.

Documento mínimo

```
{ identidad, ámbito, revisión, contrato, datos, procedencia }
```

> La forma final cambia con el dominio; los campos de gobierno permanecen verificables.

CASO 2 · FALLOS

Ensayo de degradación

Fallo	Respuesta esperada
Evento duplicado	Un solo efecto por revisión
Proyección atrasada	Frescura visible y degradación declarada
Ámbito manipulado	Rechazo antes de recuperar documentos
Cambio incompatible	Nueva versión y convivencia controlada

Evidencia

Guarde manifiesto, entradas, revisión inicial y final, métricas, caso negativo y procedimiento de reversión.

CASO 2 · REVISIÓN

Preguntas para el equipo

¿Dónde vive la única autoridad de cambio?

¿Qué copia puede borrarse hoy?

¿Qué lectura tolera atraso y cuánto?

¿Qué dato no debe cruzar un ámbito?

¿Cómo se reproduce una cifra analítica?

¿Qué mecanismo podría retirarse?

Cierre. El caso termina cuando otra persona puede reconstruir la decisión sin preguntarle al autor.

CASO GUIADO 3

Gestión territorial comunitaria

Equipos de campo documentan solicitudes, visitas, evidencia y decisiones con acceso delimitado.

> **Patrones principales.** Identidad estable, Validación en frontera, Auditoría narrativa, Recuperación limitada y Abstención con evidencia.

CASO 3 · DISEÑO

Del lenguaje del dominio al documento

Decisiones

1. Definir identidad y ámbito antes de elegir colecciones.
2. Separar hechos históricos de estados reemplazables.
3. Versionar vocabularios y contratos desde la primera migración.
4. Diseñar cada proyección desde una consulta concreta.

Documento mínimo

```
{ identidad, ámbito, revisión, contrato, datos, procedencia }
```

> La forma final cambia con el dominio; los campos de gobierno permanecen verificables.

CASO 3 · FALLOS

Ensayo de degradación

Fallo	Respuesta esperada
Evento duplicado	Un solo efecto por revisión
Proyección atrasada	Frescura visible y degradación declarada
Ámbito manipulado	Rechazo antes de recuperar documentos
Cambio incompatible	Nueva versión y convivencia controlada

Evidencia

Guarde manifiesto, entradas, revisión inicial y final, métricas, caso negativo y procedimiento de reversión.

CASO 3 · REVISIÓN

Preguntas para el equipo

¿Dónde vive la única autoridad de cambio?

¿Qué copia puede borrarse hoy?

¿Qué lectura tolera atraso y cuánto?

¿Qué dato no debe cruzar un ámbito?

¿Cómo se reproduce una cifra analítica?

¿Qué mecanismo podría retirarse?

Cierre. El caso termina cuando otra persona puede reconstruir la decisión sin preguntarle al autor.

TALLER PRÁCTICO

Construya una arquitectura COBRIA

Ocho estaciones convierten un problema real en un diseño verificable. Trabaje con una muestra pequeña y conserve cada decisión.

ESTACIÓN 1 DE 8

Inventario de decisiones y consumidores

Producto de la estación

Mapa de escrituras, lecturas, responsables y consecuencias.

Trabajo

1. Describa el estado inicial con un ejemplo concreto.
2. Escriba el contrato antes de implementar.
3. Diseñe un caso feliz y dos negativos.
4. Defina la evidencia que conservará.
5. Revise con una persona que no participó en el diseño.

> **Criterio de avance.** No continúe si la autoridad, el ámbito o la evidencia siguen implícitos.

ESTACIÓN 2 DE 8

Diseño de documentos e identidad

Producto de la estación

Tres documentos con identidad, revisión, tiempos y procedencia.

Trabajo

1. Describa el estado inicial con un ejemplo concreto.
2. Escriba el contrato antes de implementar.
3. Diseñe un caso feliz y dos negativos.
4. Defina la evidencia que conservará.
5. Revise con una persona que no participó en el diseño.

> **Criterio de avance.** No continúe si la autoridad, el ámbito o la evidencia siguen implícitos.

ESTACIÓN 3 DE 8

Contratos, ámbito y validación

Producto de la estación

Contrato de éxito, rechazo y acceso prohibido.

Trabajo

1. Describa el estado inicial con un ejemplo concreto.
2. Escriba el contrato antes de implementar.
3. Diseñe un caso feliz y dos negativos.
4. Defina la evidencia que conservará.
5. Revise con una persona que no participó en el diseño.

> **Criterio de avance.** No continúe si la autoridad, el ámbito o la evidencia siguen implícitos.

ESTACIÓN 4 DE 8

Catálogos y evolución semántica

Producto de la estación

Una entrada versionada con vigencia, sustitución y cuarentena.

Trabajo

1. Describa el estado inicial con un ejemplo concreto.
2. Escriba el contrato antes de implementar.
3. Diseñe un caso feliz y dos negativos.
4. Defina la evidencia que conservará.
5. Revise con una persona que no participó en el diseño.

> **Criterio de avance.** No continúe si la autoridad, el ámbito o la evidencia siguen implícitos.

ESTACIÓN 5 DE 8

Proyecciones e índices

Producto de la estación

Ficha de consulta con filtros, orden, salida, frescura y presupuesto.

Trabajo

1. Describa el estado inicial con un ejemplo concreto.
2. Escriba el contrato antes de implementar.
3. Diseñe un caso feliz y dos negativos.
4. Defina la evidencia que conservará.
5. Revise con una persona que no participó en el diseño.

> **Criterio de avance.** No continúe si la autoridad, el ámbito o la evidencia siguen implícitos.

ESTACIÓN 6 DE 8

Reconstrucción, eventos y reparación

Producto de la estación

Manifiesto y prueba de equivalencia incremental frente a total.

Trabajo

1. Describa el estado inicial con un ejemplo concreto.
2. Escriba el contrato antes de implementar.
3. Diseñe un caso feliz y dos negativos.
4. Defina la evidencia que conservará.
5. Revise con una persona que no participó en el diseño.

> **Criterio de avance.** No continúe si la autoridad, el ámbito o la evidencia siguen implícitos.

ESTACIÓN 7 DE 8

Minería de datos e inteligencia artificial

Producto de la estación

Partición sin fuga, definición versionada y política de abstención.

Trabajo

1. Describa el estado inicial con un ejemplo concreto.
2. Escriba el contrato antes de implementar.
3. Diseñe un caso feliz y dos negativos.
4. Defina la evidencia que conservará.
5. Revise con una persona que no participó en el diseño.

> **Criterio de avance.** No continúe si la autoridad, el ámbito o la evidencia siguen implícitos.

ESTACIÓN 8 DE 8

Evaluación, costos y retiro

Producto de la estación

Tabla comparativa, umbrales, rúbrica y procedimiento de retiro.

Trabajo

1. Describa el estado inicial con un ejemplo concreto.
2. Escriba el contrato antes de implementar.
3. Diseñe un caso feliz y dos negativos.
4. Defina la evidencia que conservará.
5. Revise con una persona que no participó en el diseño.

> **Criterio de avance.** No continúe si la autoridad, el ámbito o la evidencia siguen implícitos.

Lenguaje COBRIA

Autoridad

Lugar autorizado para aceptar un cambio legítimo.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Ámbito

Límite organizacional o de seguridad que acompaña toda operación.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Canónico

Documento desde el cual los derivados pueden volver a producirse.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Lenguaje COBRIA

Catálogo

Vocabulario versionado que estabiliza significado.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Contrato

Condición observable de entrada, resultado y fallo.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Corte

Punto temporal o de revisión usado para reproducir un conjunto.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Lenguaje COBRIA

Derivado

Representación orientada a una lectura, analítica o IA.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Frescura

Edad observable de una proyección frente a su fuente.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Idempotencia

Propiedad por la cual repetir no añade efectos distintos.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Lenguaje COBRIA

Linaje

Enlaces entre fuente, transformación, versión y resultado.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Manifiesto

Registro sellado de materiales y parámetros de construcción.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Procedencia

Origen y transformación documentados de un dato.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Lenguaje COBRIA

Proyección

Derivado diseñado para una necesidad concreta.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Reconstrucción

Producción repetible de un derivado desde fuentes autorizadas.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Revisión

Número o marca monotónica que evita regresiones silenciosas.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Lenguaje COBRIA

Tiempo conocido

Momento en que el sistema aprendió un hecho.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Tiempo válido

Momento del mundo al que el hecho se refiere.

Pregunta: ¿dónde puede observarse y cómo se prueba?

Ventana

Atraso máximo aceptado por una lectura.

Pregunta: ¿dónde puede observarse y cómo se prueba?

ÍNDICE DE DECISIONES

¿Por dónde comienzo?

Necesidad	Primer patrón
No sé cuál copia contiene la verdad	Objeto canónico
Se cruzan organizaciones o permisos	Repositorio con ámbito
Los códigos cambian de significado	Catálogo versionado
Una consulta exige otra forma	Índice antes que proyección
No puedo rehacer una copia	Reconstrucción total
Las cifras no se reproducen	Conjunto de datos reproducible
La IA responde sin sustento	Abstención con evidencia

RÚBRICA FINAL

Evaluación de una arquitectura COBRIA

Califique de cero a dos: ausente, parcial o comprobado.

Autoridad única	0 · 1 · 2	Ámbito obligatorio	0 · 1 · 2
Identidad estable	0 · 1 · 2	Revisión monotónica	0 · 1 · 2
Contrato versionado	0 · 1 · 2	Procedencia	0 · 1 · 2
Reconstrucción	0 · 1 · 2	Frescura observable	0 · 1 · 2
Caso negativo	0 · 1 · 2	Retiro seguro	0 · 1 · 2

> Un cero en autoridad o aislamiento no se compensa con promedios altos.

FUENTES Y ANTECEDENTES

Bibliografía seleccionada

Estas obras permiten situar COBRIA frente a patrones de diseño, diseño dirigido por el dominio, sistemas de datos y bases NoSQL. Las referencias se presentan conforme a la estructura autor-fecha-título-fuente de APA 7.

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.).

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Sadalage, P. J., & Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.

Criterio editorial. Una fuente incluida aquí debe aparecer cuando sustenta una afirmación del texto; a su vez, toda cita del texto debe tener su entrada completa en la bibliografía.

CIERRE

Diseñar también es dejar un camino de regreso

COBRIA propone una disciplina sencilla de expresar y exigente de demostrar: una fuente autorizada, límites explícitos y derivados útiles que puedan explicarse, comprobarse, reconstruirse y retirarse.

Este libro contiene treinta patrones, treinta laboratorios, tres casos guiados, ocho estaciones de taller y un glosario operativo. El atlas interactivo amplía la exploración mediante búsqueda, comparación y simulación.

> **Última pregunta.** Si mañana desapareciera una proyección, ¿su equipo sabría reconstruirla sin inventar información?